

Dossier de Projet

Lexigle (.NET 8 / C#) · Migration Symfony 6.4 → 7.4

Informations candidat

Candidat Lucas BLANCHARD--TOULET

Entreprise Crédit Agricole — Lab'TP

Tuteur entreprise Adrien BOULINEAU

Centre de formation ISFAC

Année scolaire 2025 – 2026

Session de certification — 2026

Remerciements

Tout d'abord, je tiens à remercier l'ISFAC, mon centre de formation qui m'a apporté la théorie nécessaire pour comprendre et écrire du code.

Je remercie également mon lieu de stage, le Lab'TP du Crédit Agricole, qui m'a permis de me former en tant que professionnel et d'allier théorie et pratique.

Merci à Adrien Boulineau pour m'avoir accompagné durant la période de stage et d'avoir pris le temps de répondre à toutes mes questions. Merci également à Oliver Llyod pour l'aide apportée à la rédaction de ce dossier, ainsi qu'à Jean-Christophe Boursier pour tous les précieux conseils apportés.

Je souhaiterais également remercier mes camarades de promotion avec qui j'ai pu échanger tout au long de cette année, toujours dans la bonne humeur.

Je remercie toutes les personnes de mon entourage proches qui n'ont cessé de me soutenir et m'encourager tout au long de ce parcours et me permettre d'aller jusqu'au bout de ma formation.

Table des matières

Remerciements.....	2
Introduction.....	5
Présentation du cadre professionnel.....	6
1. Présentation de l'entreprise : Le Lab'TP du Crédit Agricole.....	6
2. Contexte et objectifs des projets	6
Le sujet principal de mon stage (projet de développement piloté de A à Z) : Lexigle (.NET/C#).....	6
Le sujet secondaire (Migration Symfony 6,4 → 7,4) :.....	7
3. Le pilotage d'un projet au sein du crédit agricole.....	7
1. La qualification et l'étude du besoin.....	7
2. Cadrage et spécifications.....	7
3. Développement par lots et automatisation.....	8
4. Recette (UAT) et état actuel du projet.....	8
Présentation du projet et du contexte.....	9
1. Présentation du projet : Lexigle.....	9
Le contexte et le besoin.....	9
La solution proposée.....	9
Mon rôle sur le projet.....	9
2. Fonctionnalités.....	10
2.1 ROLE_UTILISATEUR (Tous Collaborateurs CATP).....	10
2.2 ROLE_ADMIN_METIER (Référént fonctionnel).....	10
2.3 ROLE_ADMIN_TECH (Maintenance).....	10
3. Organisation du projet et livraison continue.....	11
Lot 1 : Cœur du système et gestion des définitions.....	11
Lot 2 : Enrichissement et relations de données.....	11
Lot 3 : Gouvernance et collaboration.....	11
4. Stack Technique et Choix technologiques.....	12
4.1 Les choix technologiques.....	12
4.2 Les outils de développement.....	12
5. Environnement de développement et Architecture Blanche (CP1).....	13
5.1. Le concept d'Architecture Blanche au Lab'TP.....	13
5.2. Appropriation et montée en compétence.....	13
5.3 Les concepts déjà implémentés.....	13
Liste des compétences couvertes par les projets.....	14
1. Développer la partie front-end d'une application web ou web mobile sécurisée.....	14
1.1 Installer et configurer son environnement de travail en fonction du projet web et web mobile.....	14
1.2 Maquetter des interfaces utilisateur web ou web mobile.....	14
1.3 Réaliser des interfaces utilisateur statiques web ou web mobile.....	14
1.4 Développer la partie dynamique des interfaces utilisateur web ou web ou web mobile.....	14
2. Développer la partie back-end d'une application web ou web mobile sécurisée.....	14
2.1 Mettre en place une base de données relationnelle.....	14
2.2 Développer des composants d'accès aux données SQL et NoSQL. .	15
2.3 Développer des composants métiers coté serveur.....	15
2.4 Documenter le déploiement d'une application dynamique web ou web mobile.....	15
Architecture technique du projet.....	16
L'utilité d'une architecture multicouche ou découplée.....	16
Flux de données et rôle des objets (DTO).....	16

Le contrat et l'implémentation (découpage Domaine/WebApp).....	17
Externalisation du mapping grâce aux méthodes d'extension.....	17
Réalisations front-end.....	18
Maquettes et parcours utilisateur (CP2).....	18
Page d'accueil.....	18
Les résultats.....	19
Les formulaires.....	21
Gestion des alertes.....	22
Le schéma d'enchaînement.....	23
Réaliser des interfaces utilisateurs statiques (CP3).....	24
Page d'accueil.....	24
Formulaire de création.....	25
La modal d'information.....	26
L'envoi de mail pour suggérer une modification.....	27
Développer la partie dynamique des interfaces web (CP4).....	28
La recherche : les suggestions autocomplétée.....	28
L'ajout des mots-clés avec Tom-Select.....	30
Flash message dynamiquement et externalisé.....	31
Réalisation back-end.....	32
Mettre en place une base de données relationnelle (CP5).....	32
Conception initiale et évolution.....	32
Conception retenue.....	32
Persistances et migrations.....	34
Développer des composants d'accès aux données SQL (CP6).....	36
La chaîne de connexion et l'injection.....	36
Extraction et requêtage asynchrone (Lecture).....	36
Du modèle métier vers l'affichage (Présentation).....	36
Traitement des flux d'écriture (Création d'une définition).....	37
A. Le mapping initial (Interface → Service) :.....	37
B. Le contrôle d'unicité en base de données :.....	37
C. La projection finale vers la persistance :.....	38
Traitement des flux de mise à jour (Modification d'une définition).....	38
A. Requête d'extraction pour modification :.....	38
B. Le mapping de modification et purge des relations :.....	39
Développer des composants métier coté serveur (CP7).....	40
1. Gestion dynamique des paramètres métiers et sécurisation (Le Vault)	40
2. Injection de dépendances et isolation des responsabilités.....	41
3. Traitement et sécurisation d'une action serveur (Scénario de	42
modification).....	42
4. Traitement textuel standardisé (Le Helper).....	44
5. Validation de la qualité du code (Tests Unitaires xUnit).....	45
Documenter le déploiement d'une application (CP8).....	47
1. L'Intégration Continue (CI).....	47
2. Le Déploiement Continu (CD).....	47
CONTEXTE COMPLÉMENTAIRE : INITIATION À LA MIGRATION DE SITES	
(SYMFONY).....	49
1. Problématique d'infrastructure et contraintes de sécurité.....	49
2. Travaux réalisés et suspension du chantier.....	49
Conclusion.....	50
Un bilan projet formateur.....	50
Ce que ce stage m'a apporté.....	50
Perspectives et projets d'avenir.....	51

Introduction

D'aussi loin que je me souviens, j'ai toujours été curieux et intéressé par les technologies : les consoles, les jeux vidéo, les ordinateurs de mes parents puis le mien, et surtout Internet. Mais je n'avais jamais envisagé qu'il soit possible d'en faire un métier.

Mon parcours a été assez varié. J'ai essayé différentes voies professionnelles, sans vraiment trouver ce qui me correspondait. C'est finalement en m'intéressant un peu plus à l'informatique, et notamment au développement web, que j'ai découvert un domaine qui me passionnait.

Le déclic est venu quand, avec des amis, nous avons eu l'idée de créer une application, or aucun de nous ne savait comment s'y prendre. Alors j'ai commencé à apprendre par moi-même, en suivant des tutoriels et en explorant les bases du développement. Très vite, j'y ai pris goût.

C'est à ce moment-là que j'ai décidé de me former sérieusement dans ce domaine, pour mieux comprendre et progresser. J'ai donc intégré une formation en développement web, à l'ISFAC. J'ai commencé à m'intéresser davantage à cet univers, et grâce à mes professeurs j'ai découvert des conférences intéressantes telles que l'AFUP (Association Française des Utilisateurs PHP) ou PWN (Poitiers Web Nerds).

Dans le cadre de cette formation, j'ai effectué un stage au sein du pôle développement du Crédit Agricole de la Touraine et du Poitou. Le projet que je présente dans ce dossier a été réalisé pendant ce stage, et s'inscrit directement dans mon parcours de formation.

Présentation du cadre professionnel

1. Présentation de l'entreprise : Le Lab'TP du Crédit Agricole.

Le Crédit Agricole est un groupe national qui est divisé en 49 Caisses Régionales, dont celle de la Touraine et du Poitou où j'ai effectué mon stage.

Le groupe compte également des entités et coopérations qui regroupent des activités spécifiques mises en œuvre pour toutes les caisses régionales. L'une de ces entités s'appelle CATS (Crédit Agricole Technologie Services) et pilote la mise en place de la majorité des outils informatiques bancaires.

Les caisses régionales ont cependant des besoins spécifiques qui ne sont pas couverts par les entités du groupe. Pour répondre à ces besoins, elles ont l'option de faire développer par des prestataires ou par des équipes internes qui sont appelées « équipes péri informatiques ». Cette équipe à la caisse Touraine Poitou s'appelle le Lab'TP (Laboratoire Touraine Poitou).

L'équipe conçoit, développe et maintient principalement des applications web en **C#/I.NET** et en **PHP/Symfony**, et accompagne les collaborateurs autour des solutions qu'elle met à leur disposition.

Dans le cadre de mon stage, j'ai été amené à découvrir différentes facettes du métier de développeur dans le contexte d'une entreprise dont le développement informatique n'est pas le modèle économique (banque).

2. Contexte et objectifs des projets

Le sujet principal de mon stage (projet de développement piloté de A à Z) : Lexigle (.NET/C#)

On m'a demandé de piloter un projet de A à Z pour la création d'un glossaire interactif des acronymes métiers du Crédit Agricole en C#/I.NET8. L'outil, qui s'appelle *Lexigle*, doit permettre la mise en place d'une interface intuitive pour rechercher, ajouter et modifier des définitions avec leurs acronymes. Ce projet intègre chaque étape de la méthodologie de pilotage de projet de développement informatique du CATP. Je suis intervenu sur la **captation du besoin**, la **conception** avec la réflexion sur l'expérience utilisateur, la **modélisation** de la base de données, la rédaction d'un **cahier des charges** ainsi que le **maquettage** de l'application par wireframes, le **développement** des lots et les différentes **livraisons**. Enfin, le **déploiement** sur un serveur de **recette utilisateur** (UAT) a permis de valider le bon respect du cahier des charges avant le déploiement en **production** pour que l'outil soit accessible à tous.

3. Développement par lots et automatisation

Le projet a été exécuté en suivant le découpage par lots fonctionnels établi. Durant cette phase, j'ai travaillé en autonomie sur mes développements. Pour sécuriser le code et valider sa conformité technique avant toute phase de recette, j'ai rédigé des tests et je me suis appuyé sur les pipelines CI/CD (Intégration Continue / Déploiement Continu) déjà existants et configurés au sein de la structure.

4. Recette (UAT) et état actuel du projet

À l'issue de mon stage, le projet se trouve dans la configuration suivante :

- **Le Lot 1** (le cœur de l'application) a été entièrement finalisé, testé et livré.
- **Le Lot 2** est techniquement très avancé mais n'est pas encore totalement finalisé.

Actuellement, l'application est déployée dans un environnement de recette (**UAT - User Acceptance Testing**). Cela permet aux utilisateurs de tester et de valider les fonctionnalités du Lot 1, en attendant la finalisation des développements restants.

Présentation du projet et du contexte

| 1. Présentation du projet : Lexigle

Durant mon stage, j'ai eu l'opportunité de piloter de A à Z le projet **Lexigle** : un glossaire interactif d'acronymes métiers, développé en **C# / .NET 8**.

Le contexte et le besoin

Le secteur bancaire utilise une quantité importante de termes techniques et de sigles complexes. Cela peut rendre l'intégration des nouveaux arrivants difficile et freiner la communication entre les différents services. L'objectif de Lexigle est d'offrir aux collaborateurs du Crédit Agricole de la Touraine et du Poitou un référentiel commun pour centraliser les définitions et clarifier la communication.

La solution proposée

L'application repose sur deux piliers :

- **L'accessibilité** : Une barre de recherche centrale pour retrouver instantanément un acronyme, sur le modèle d'un moteur de recherche classique.
- **L'aspect collaboratif** : Un système de contribution permettant à chaque collaborateur connecté d'enrichir le référentiel avec de nouvelles définitions.

Mon rôle sur le projet

Ce projet me permet de maîtriser l'ensemble du cycle de vie d'une application :

- **Conception** : Réflexion sur l'expérience utilisateur (UX) et modélisation de la structure de données et des relations entre les entités.
- **Développement** : Création de l'interface et mise en place des fonctionnalités (recherche, ajout, modification).
- **Déploiement** : Mise en service d'un outil fonctionnel via une chaîne CI/CD.

2. Fonctionnalités

L'application *Lexigle* s'articule autour de trois rôles, auxquels sont attribuées des fonctionnalités correspondant au niveau d'habilitation du rôle.

2.1 ROLE_UTILISATEUR (Tous Collaborateurs CATP)

- **Rechercher** : Trouver une définition via l'acronyme, son libellé ou des tags associés.
- **Consulter** : Accéder au détail complet d'un terme et visualiser les définitions liées entre elles (références croisées).
- **Contribuer** : Proposer de nouveaux termes ou suggérer des modifications/suppressions sur des fiches existantes (avec justification obligatoire).

La fonctionnalité de suggestion peut être désactivée. Dans ce cas-là, les utilisateurs finaux pourront poster, modifier ou supprimer directement leurs propres définitions, mais toute trace de leurs actions sera conservée.

2.2 ROLE_ADMIN_METIER (Référent fonctionnel)

Ce rôle est attribué au service garant de la qualité du glossaire. Ses missions sont :

- **Modérer** : Accepter ou refuser les suggestions soumises par les utilisateurs.
- **Gérer le contenu** : Créer, modifier ou supprimer n'importe quelle définition pour maintenir la base à jour.

2.3 ROLE_ADMIN_Tech (Maintenance)

Ce rôle est réservé au pôle **Lab'TP**. Il assure la maintenance technique, la gestion des paramètres système de l'application et le suivi du bon fonctionnement de la plateforme.

3. Organisation du projet et livraison continue

Le développement de *Lexigle* suit une approche agile, segmentée en trois lots fonctionnels. Chaque lot fait l'objet d'un déploiement sur l'environnement d'UAT (User Acceptance Test) pour validation avant d'entamer la phase suivante.

Lot 1 : Cœur du système et gestion des définitions

Ce premier lot constitue le socle technique de l'application.

- **Fonctionnalités** : Mise en place du CRUD complet (Create, Read, Update, Delete) sur les définitions et moteur de recherche principal.
- **Technique** : Déploiement de l'architecture globale, sécurisation des accès et implémentation des tests unitaires et fonctionnels pour garantir la stabilité de la solution.

Lot 2 : Enrichissement et relations de données

Le deuxième lot se concentre sur l'organisation et l'interconnectivité des données.

- **Gestion des Tags** : Implémentation du système de tags (CRUD et recherche filtrée) pour faciliter le classement des acronymes.
- **Relations croisées** : Mise en place d'une relation **auto-référencée (N,N)** permettant de lier plusieurs définitions entre elles, offrant ainsi une navigation contextuelle aux utilisateurs.

Lot 3 : Gouvernance et collaboration

Le dernier lot apporte les couches de modération et de gestion des accès.

- **Workflow de suggestion** : Développement du système de soumission et de validation des modifications par les utilisateurs.
- **Droits et administration** : Finalisation de la gestion des rôles (Admin Métier / Admin Tech) et mise en place des outils de modération pour garantir l'intégrité du référentiel.

4. Stack Technique et Choix technologiques

4.1 Les choix technologiques

Le choix des technologies pour *Lexigle* a été guidé par les standards internes du Lab'TP, afin d'assurer une intégration fluide et une maintenance simplifiée au sein de l'entreprise.

- **Back-end (C# / .NET 8)** : Le projet est développé sous **.NET 8** pour maintenir une cohérence avec le parc applicatif interne du Lab'TP, tout en bénéficiant d'un framework moderne et performant.
- **Accès aux données (EF Core & SQL Server)** : J'utilise l'ORM **Entity Framework Core** pour la gestion de la persistance, couplé à une base de données **SQL Server**. Ce duo permet une manipulation des données sécurisée et une gestion simplifiée des migrations.
- **Front-end (Razor & Bootstrap)** : L'interface est générée via le moteur de templates **Razor**, intégré à ASP.NET Core, assurant un rendu serveur robuste. Le design et le responsive sont gérés par le framework **Bootstrap**.
- **Dynamisme (Stimulus JS)** : Pour l'interactivité (recherche instantanée, gestion des formulaires), j'ai implémenté **Stimulus**. Ce framework JavaScript léger a été choisi pour sa simplicité et sa parfaite intégration au rendu côté serveur, évitant la lourdeur d'une SPA (Single Page Application).

4.2 Les outils de développement

Pour mener à bien le développement de *Lexigle* au sein du Lab'TP, j'ai utilisé un ensemble d'outils professionnels permettant de garantir la qualité du code et le suivi du projet :

- **IDE (Environnement de développement)** : J'ai travaillé principalement sur **Visual Studio 2022**. C'est l'outil de référence pour le développement .NET, offrant des fonctionnalités avancées de débogage et une intégration parfaite avec le socle applicatif fourni.
- **Gestionnaire de version** : Le versionnage du code est assuré par **Git**, avec un hébergement sur les serveurs GitLab du Crédit Agricole. Cela m'a permis de travailler par "branches" et par versions afin de sécuriser chaque étape de mon développement.

5. Environnement de développement et Architecture Blanche (CP1)

5.1. Le concept d'Architecture Blanche au Lab'TP

Le développement de Lexigle ne s'est pas fait en partant d'un projet vide. J'ai démarré le projet en intégrant mon travail à une "**Architecture Blanche**" (ou "Template Entreprise") conçue par le pôle Lab'TP. Il s'agit d'un socle applicatif standardisé qui sert de base à tous les nouveaux projets internes du Crédit Agricole Touraine-Poitou. Ce socle impose une structure rigoureuse et intègre déjà des fonctionnalités clés :

- **La sécurité et l'authentification.**
- **La gestion des journaux (Logs).**
- **La configuration des accès aux bases de données.**

5.2. Appropriation et montée en compétence

Ma première mission a été de m'approprier ce code existant. Contrairement à un projet scolaire où l'on maîtrise chaque ligne dès le début, j'ai dû apprendre à lire et à comprendre une architecture experte pour y greffer mes fonctionnalités sans rompre la logique du système.

Cette étape, bien que complexe au démarrage, a été très formatrice : elle m'a permis de comprendre comment sont structurées des applications industrielles.

5.3 Les concepts déjà implémentés.

L'application blanche fournie par le Lab'TP a eu une influence sur ma conception de la base de données. En effet, je n'ai pas eu à m'occuper de l'authentification des utilisateurs ni de l'entité « Utilisateur » car les projets du Crédit Agricole implémentent une base de données commune qui fournit toutes les informations sur les collaborateurs du groupe.

L'authentification se fait donc par **SSO (Single Sign-On)**. L'application identifie l'utilisateur via sa session Windows active. Cela garantit une sécurité maximale et une expérience fluide pour le collaborateur, qui n'a aucun mot de passe à saisir.

Liste des compétences couvertes par les projets

1. Développer la partie front-end d'une application web ou web mobile sécurisée

1.1 Installer et configurer son environnement de travail en fonction du projet web et web mobile

Le Crédit Agricole m'a fourni un ordinateur portable prêt à l'emploi pour le développement, avec déjà les logiciels installés, car une fois en ma possession j'étais limité par un proxy pour les téléchargements. Pour l'application en .NET, mon maître de stage a créé une « application blanche » qui doit être la base de toutes les futures applications qui seront sur leurs serveurs.

1.2 Maquetter des interfaces utilisateur web ou web mobile

J'ai réalisé à l'aide d'Excalidraw un wireframe des différents écrans que l'utilisateur empruntera lors de ses interactions avec l'application.

1.3 Réaliser des interfaces utilisateur statiques web ou web mobile

L'application *Lexigle* est un glossaire interactif et collaboratif, de ce fait, très peu de pages sont purement statiques. Cependant j'ai réalisé des visuels avec des parties statiques en *HTML*, *SCSS* et avec le framework *Bootstrap*.

1.4 Développer la partie dynamique des interfaces utilisateur web ou web mobile

Pour la partie dynamique du site, j'ai utilisé le framework *Stimulus* pour les animations et les actions, pour un appel à une API *Fetch* et le composant *Tom-Select*. Le rendu des données dans les templates est réalisé grâce à *Razor*.

2. Développer la partie back-end d'une application web ou web mobile sécurisée

2.1 Mettre en place une base de données relationnelle

J'ai dans un premier temps réfléchi à la conception de la base de données en fonction des besoins de l'application. J'ai utilisé *Excalidraw* pour maquetter mes différentes entités et les relations qu'elles auront entre elles, que j'ai ensuite créées et migrées directement depuis mon environnement de développement vers une base *SQL Server*.

2.2 Développer des composants d'accès aux données SQL et NoSQL

L'application intègre un moteur de recherche et un dictionnaire de définitions qui nécessitent une communication constante et optimisée avec une base de données SQL Server. À travers l'utilisation d'un ORM et de requêtes structurées, le projet met en œuvre tous les composants indispensables à la sélection, au filtrage et à l'extraction sécurisée des données métiers.

2.3 Développer des composants métiers côté serveur

La couche Web (MVC) et la couche Domaine de l'application embarquent toute la logique applicative et technique nécessaire. Du contrôle de la sécurité des accès à la validation des formulaires, en passant par le nettoyage standardisé des saisies et les tests de non-régression, le projet démontre une gestion complète et structurée des traitements côté serveur.

2.4 Documenter le déploiement d'une application dynamique web ou web mobile

Pour ce projet, j'utilise une chaîne CI/CD industrielle adaptée aux exigences du secteur bancaire. À chaque « push » sur le dépôt, un pipeline d'intégration continue assure automatiquement la compilation et l'analyse du code via SonarQube et Xray (contrôle du *code coverage*). Le déploiement (CD) vers l'environnement d'UAT est préconfiguré et déclenché manuellement après validation de la qualité logicielle. Ce processus permet de livrer l'application de manière sécurisée et maîtrisée sur le serveur de test.

Architecture technique du projet

L'utilité d'une architecture multicouche ou découplée

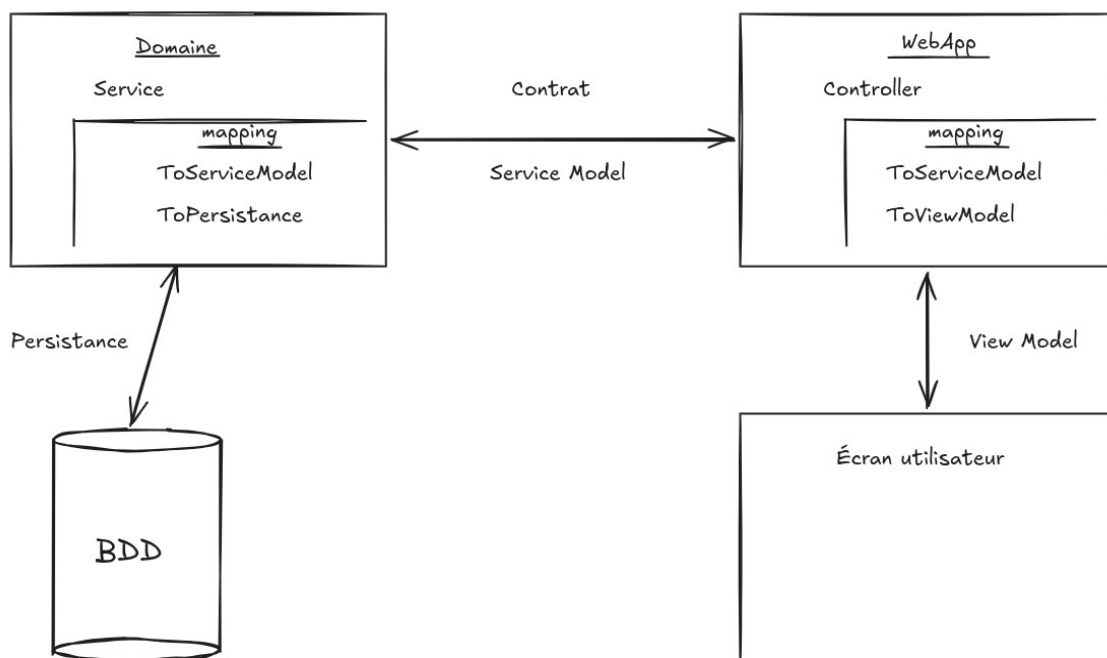
L'architecture multicouche me permet d'assurer la maintenabilité de *Lexigle*. Contrairement à une architecture « monolithique » où tout le code est mélangé, le découpage consiste à séparer l'application en blocs de responsabilités distincts.

L'utilité de cette approche est triple :

- 1 **La sécurité des données (Encapsulation)** : C'est un point critique, particulièrement dans un environnement bancaire. Le découpage permet de ne jamais exposer la structure réelle de la base de données à l'interface utilisateur. En utilisant des objets intermédiaires, on crée un filtre : l'utilisateur ne reçoit que les données strictement nécessaires à l'affichage. Cela empêche l'exposition accidentelle de données sensibles ou techniques qui pourraient être exploitées pour comprendre l'organisation du Système d'Information.
- 2 **La testabilité** : Chaque couche peut être testée indépendamment, ce qui garantit un code plus robuste et moins sujet aux régressions.
- 3 **La clarté du développement** : Elle permet de bien séparer la logique métier de la logique d'affichage.

Flux de données et rôle des objets (DTO)

Le schéma ci-dessous illustre comment les données circulent et se transforment au sein de l'application, de la persistance jusqu'à l'écran final.



Dans ce schéma, on distingue trois modèles de la donnée :

1. **La Persistance (Modèle de données)** : L'image exacte des tables en base de données.
2. **Le Service Model** : Définis dans la couche **Domaine**, c'est une version épurée et sécurisée de la donnée, utilisée pour les traitements métiers.
3. **Le View Model** : Définis dans la couche **WebApp**, c'est un objet "sur-mesure" pour l'affichage, contenant uniquement les données nécessaires à la génération du templates.

Le contrat et l'implémentation (découpage Domaine/WebApp)

La communication entre l'interface (WebApp) et la logique métier (Domaine) repose sur un principe de **Contrat** :

- **L'Interface (Le Contrat)** : C'est un fichier qui liste les fonctionnalités disponibles (ex. : `GetDernieresDefinitions()`). Le Controller appelle l'interface sans savoir comment le code est écrit derrière.
- **Le Service (L'Implémentation)** : C'est ici que j'ai rédigé les requêtes **LINQ**. C'est le moteur de l'application. Il récupère les données brutes, effectue les tris ou les calculs, et les transforme en *Service Models*.

Externalisation du mapping grâce aux méthodes d'extension

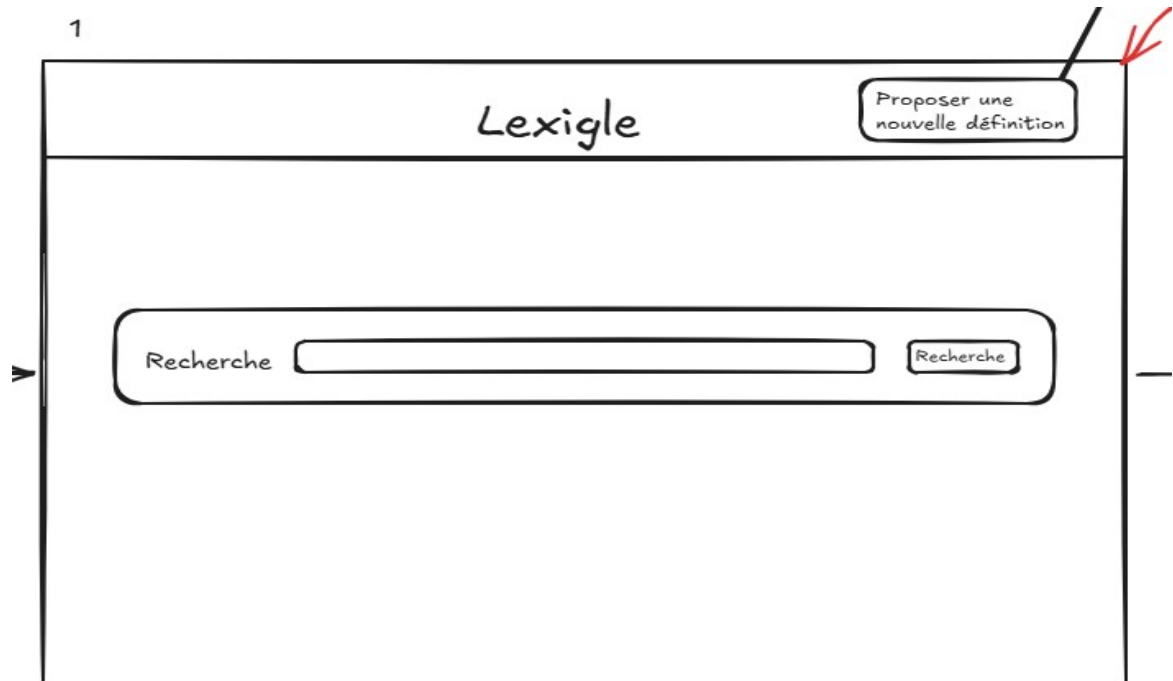
Afin d'assurer une conversion fiable entre les différents modèles, j'ai appliqué l'usage des méthodes d'extension C# à chaque étape du flux de données. Ce mapping externalisé garantit une étanchéité entre les couches du projet en isolant les transformations techniques dans des fichiers dédiés.

Cette approche permet de conserver des contrôleurs légers et lisibles. Plutôt que de manipuler des constructeurs complexes au cœur de la logique métier, le Controller et le Service utilisent une syntaxe naturelle en appelant simplement ces méthodes d'extension. Cette organisation facilite grandement la maintenance technique : toute modification de structure reste confiée aux fichiers de mapping, préservant ainsi l'intégrité et la stabilité du code dans les autres couches de l'application.

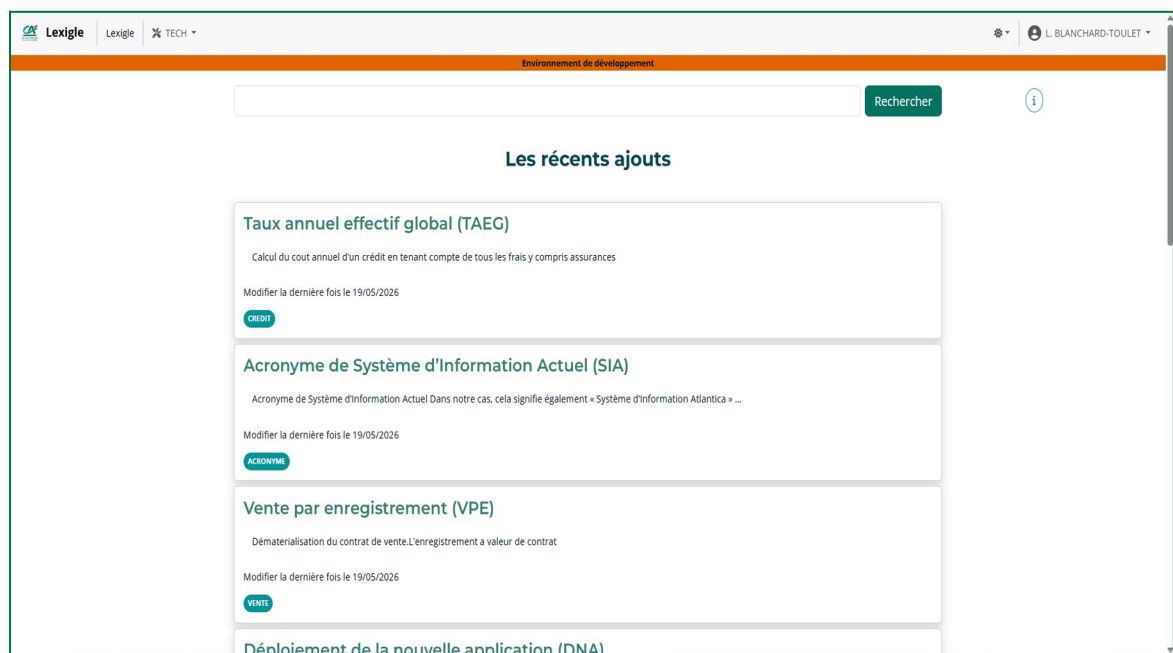
Réalisations front-end

Maquettes et parcours utilisateur (CP2)

Page d'accueil



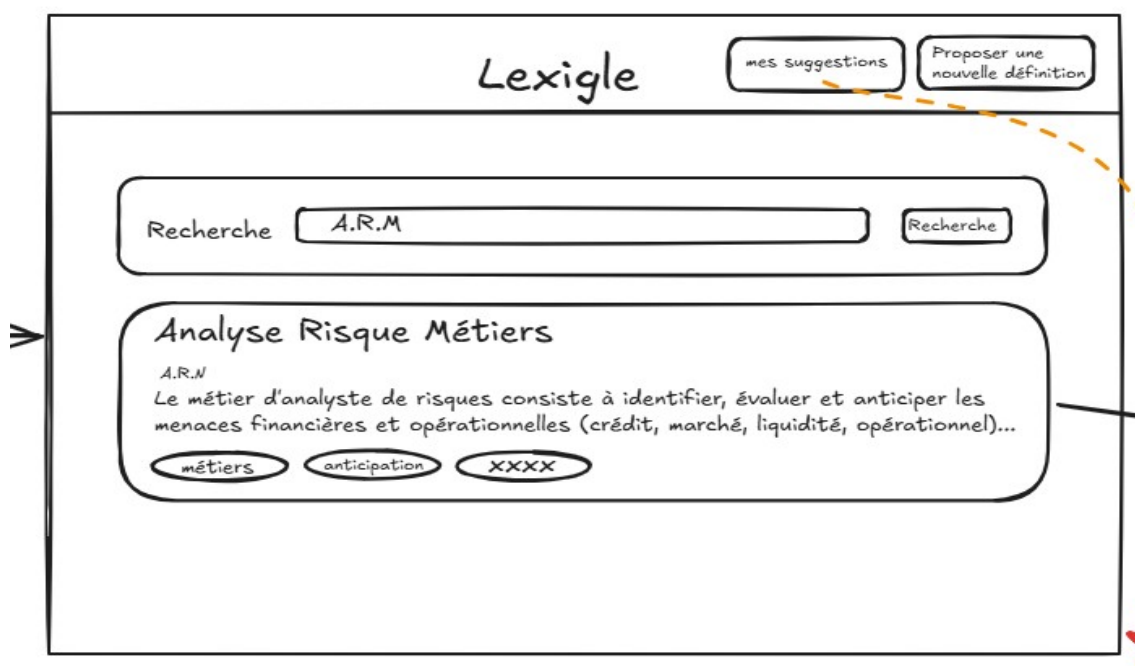
L'objectif premier était la simplicité. Je voulais un champ de recherche central, très large, pour que l'utilisateur n'ait aucune hésitation en arrivant sur l'application.



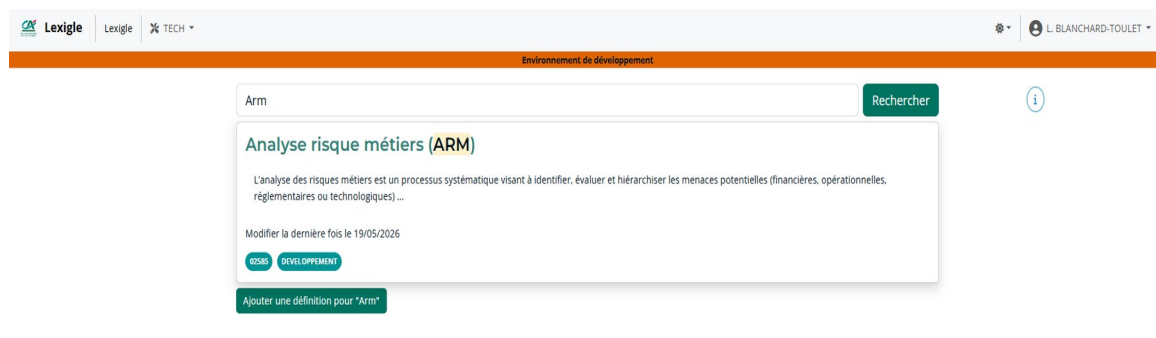
Le rendu final intègre la charte graphique du Crédit Agricole (couleur, typographies). J'ai conservé la structure du wireframe tout en ajoutant une liste des 10 derniers éléments (valeur modifiable). Cet ajout permet de prouver visuellement que le glossaire est enrichi régulièrement, mais également d'offrir un accès immédiat aux

termes les plus récents.

Les résultats



L'objectif de cette vue était de permettre une lecture rapide parmi une liste de résultats potentiellement nombreux. J'ai conçu cette interface pour qu'elle affiche l'essentiel au premier regard : le libellé en titre, les acronymes associés à cette définition juste en dessous, le début de la description pour donner du contexte, puis une liste de tags. La structure en « liste de carte » a été choisie pour faciliter le scan visuel, permettant à l'utilisateur de confirmer rapidement s'il a trouvé le bon terme sans avoir à cliquer sur chaque fiche.



Le rendu final est globalement resté fidèle au wireframe, à quelques exceptions près : chaque définition n'a désormais plus qu'un acronyme associé, il est donc marqué avec le libellé dans le titre. Le bouton « ajouter une définition » s'affiche désormais seulement après une recherche, et est prérempli par le terme recherché. Cette modification permet de forcer l'utilisateur à rechercher une définition avant de pouvoir en ajouter, évitant ainsi les définitions en doublons.

Techniquement, cette vue s'appuie sur une boucle qui parcourt les DTO de recherche pour générer dynamiquement les composants. Si aucun résultat n'est trouvé alors le texte « Aucun résultat » s'affiche. J'ai utilisé les « cards » de **Bootstrap** pour assurer une fluidité visuelle et une séparation nette entre chaque résultat.

L'interface de la définition détaillée présente à l'utilisateur l'intégralité des informations associées au terme sélectionné. Visuellement, le composant est structuré pour afficher de manière hiérarchisée le libellé principal, l'acronyme correspondant ainsi que la définition textuelle complète.

Il donne aussi accès au formulaire de modification si l'utilisateur en a les droits. Si ce n'est pas le cas, j'ai rajouté à la version finale un lien pour envoyer directement un e-mail à l'auteur de la définition pour lui soumettre des modifications.

Lexigle

Analyse Risque Métiers

Le métier d'analyste risques consiste à identifier, évaluer, mesurer et anticiper les différents types de risques auxquels une entreprise (banque, assurance, cabinet de conseil) est exposée, en veillant à l'équilibre entre la rentabilité et la conformité réglementaire. Ce professionnel, souvent spécialisé (risques de crédit, de marché, de liquidité ou opérationnels), réalise des études, des simulations et met en place des outils de notation et de surveillance pour protéger l'organisation.

A.R.N - X.X.X

métiers

anticipation

XXXX

rédigé par Jean. Dupont

Retour

Modifier

Lexigle

Lexigle

TECH

Environnement de développement

L. BLANCHARD-TOULET

Libelle : Analyse risque métiers

Acronyme : ARM

L'analyse des risques métiers est un processus systématique visant à identifier, évaluer et hiérarchiser les menaces potentielles (financières, opérationnelles, réglementaires ou technologiques) liées à la croissance, à l'expansion ou à la création de nouveaux produits au sein d'une entreprise. Cette démarche permet de quantifier l'impact de ces risques sur la rentabilité et la stabilité de l'organisation, afin de mettre en place des stratégies de prévention, de mitigation ou de transfert appropriées (avec la méthode MESARI).

Mots-clés

OSI

DEVELOPPEMENT

Modifier la dernière fois le 19/05/2026

Par : BLANCHARD TOULET LUCAS

Vous pouvez suggérer une modification en envoyant un mail à Lucas.BLANCHARD-TOULET@ca-touraineoitou.fr

Retour

Modifier

Lucas BLANCHARD-TOULET

20 / 51

Les formulaires

Les formulaires de création et de modification sont visuellement similaires pour garder une cohérence dans l'expérience utilisateur lors d'une création ou d'une suggestion de modification.

2

The wireframe shows a form titled 'Lexigle'. At the top, it asks 'Voulez-vous lier à une autre définition ?' with radio buttons for 'non' (checked) and 'oui'. Below this is a dropdown menu with 'ARM' selected and 'Analyse Risque Métiers' as a suggestion. The form then has fields for 'Acronyme :', 'Définition :', 'Tag(s) :', and 'Description :'. The 'Tag(s) :' field shows 'métiers' as a selected tag and a list of suggestions: 'anticipation', 'anticipation', and 'anticoagulant'. At the bottom, there are three buttons: 'Retour' (red), 'Soumettre à validation' (green), and 'Supprimer' (red).

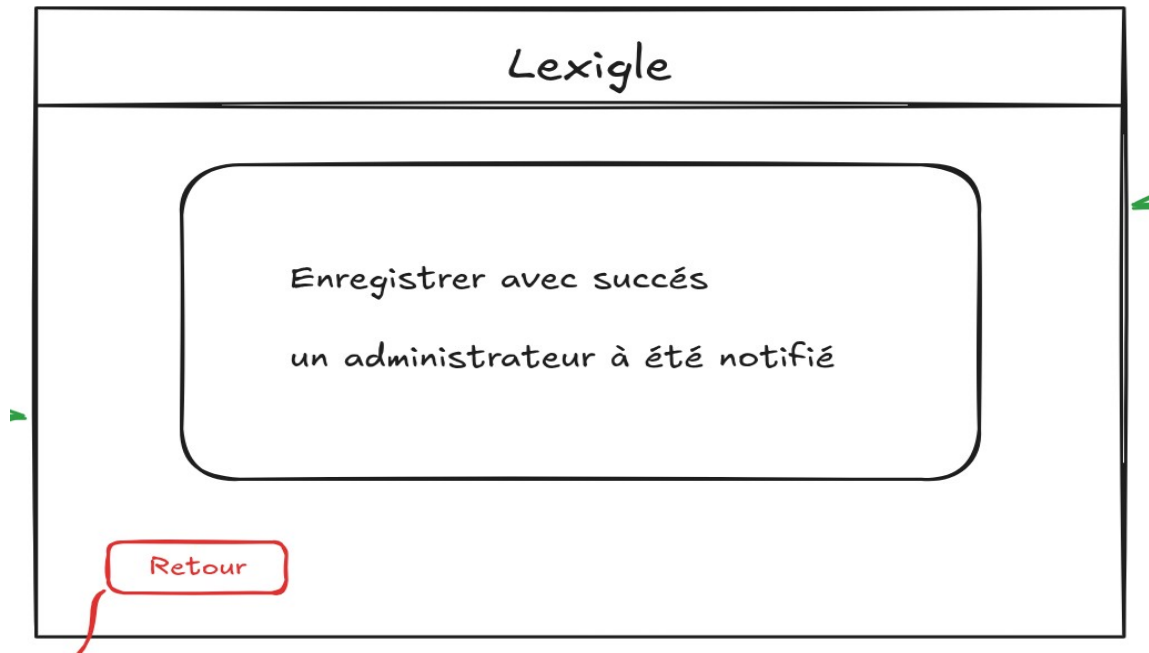
Le but de ce wireframe était de structurer la saisie pour qu'elle soit la plus fluide possible. Les informations importantes sont regroupées ensemble (Acronyme / Libellé, les mots-clés) puis la description en dessous. L'ajout d'un bouton radio au-dessus permettait au départ d'ouvrir un panel qui nous donne la possibilité de lier cette définition à une autre.

The screenshot shows the 'Lexigle' application interface. The header includes the 'Lexigle' logo, a 'Lexigle' tab, a 'TECH' dropdown, and a user profile 'L. BLANCHARD-TOULET'. The main content area is titled 'Environnement de développement' and contains a form with the following fields: 'Libellé' (containing 'Application Programming Interface'), 'Acronyme' (containing 'api'), 'Mots-clés' (containing '02585', 'DEVELOPPEMENT', and 'Mots-clés'), and 'Description' (containing a text about API). At the bottom, there are two buttons: 'Retour' and 'Soumettre ma proposition'.

Le formulaire final reprend cette structure en y ajoutant la gestion dynamique des données.

- **En mode Ajout** : Le champ acronyme est automatiquement prérempli avec le terme issu de la recherche précédente pour éviter les erreurs de saisie. Le bouton « switch » permet de changer la valeur entre l'acronyme et le libellé.
- **En mode Modification** : L'application vient "mapper" les données existantes de la base fournies par le DTO dans les champs du formulaire pour permettre une édition rapide.

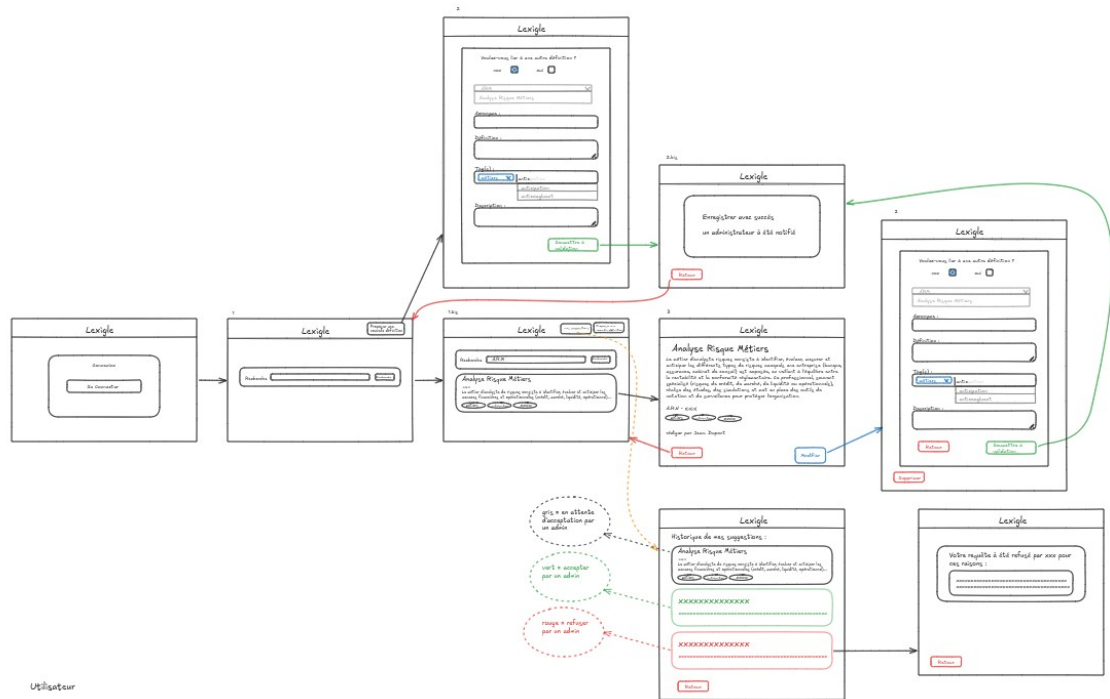
Gestion des alertes



Initialement pensée comme une vue à part pour informer les utilisateurs en cas de succès ou d'échec de validations des formulaires, j'ai finalement opté pour des flash messages qui apparaissent sur l'index avec les classes Bootstrap.



Le schéma d'enchaînement



Ce schéma présente l'enchaînement des écrans de l'application Lexigle du point de vue d'un utilisateur connecté. Il illustre les deux parcours principaux : la recherche d'une définition existante et la contribution via le formulaire d'ajout.

Réaliser des interfaces utilisateurs statiques (CP3)

L'application blanche fournit directement une structure MVC, avec un layout global qui possède déjà le header, le footer et le conteneur selon la charte du Crédit Agricole. Par conséquent, mes vues (.cshtml) se concentrent uniquement sur le cœur de page. Pour le développement de ces interfaces, j'ai utilisé la version de **Bootstrap personnalisée** de l'entreprise. Cela m'a permis d'appliquer directement la palette de couleurs et la typographie de la charte et de gérer le responsive (mobiles, tablettes, PC) en utilisant le système de grille et les classes utilitaires du framework, sans avoir à réécrire du CSS spécifique.

Page d'accueil

```
@* BARRE DE RECHERCHE *@
<form asp-action="Rechercher" method="get" class="mt-4">
    <div class="row g-2">
        <div class="col position-relative" data-controller="recherche" data-recherche-url-value="@Url.Content("~/Lexigle/Suggestions")">
            <input type="text" asp-for="Recherche" class="form-control form-control-lg" data-recherche-target="input" data-action="input->recherche#suggestion" />
            <ul data-recherche-target="suggestionBox" class="list-group position-absolute w-100 z-3 shadow"></ul>
        </div>
        <div class="col-auto">
            <button type="submit" class="btn btn-lg btn-primary">
                Rechercher
            </button>
        </div>
    </div>
</form>

@* AFFICHAGE DEFINITIONS *@
@if (Model.Definitions.Any())
{
    @if (string.IsNullOrEmpty(Model.Recherche))
    {
        <h2 class="my-5 text-center">Les récents ajouts</h2>
    }

    @if (Model.Definitions.Count >= 4 && !string.IsNullOrEmpty(Model.Recherche))
    {
        @* BTN CREATION *@
        <a class="btn btn-primary mt-3" asp-action="CreationDefinition" asp-route-libelle="@Model.Recherche">
            Ajouter une définition pour "@Model.Recherche"
        </a>
    }

    foreach (DefinitionViewModel item in Model.Definitions)
    {
        <a asp-action="Definition" asp-route-id="@item.Id" class="text-decoration-none">
            <div class="card my-2 p-3 shadow" data-controller="highlight" data-highlight-term-value="@Model.Recherche">
                <h3 class="card-title">@item.Libelle @(item.Acronyme?.Length > 0 ? "(" + item.Acronyme + ")" : " ")</h3>
                <p class="card-body">@item.Description.Truncate(Model.MaxLenthTroncate)</p>

                @if (item.DateModification > item.DateCreation)
                {
                    <p>Modifier la dernière fois le @item.DateModification.ToString("dd/MM/yyyy")</p>
                }
                else
                {
                    <p>Ajouter le @item.DateCreation.ToString("dd/MM/yyyy")</p>
                }

                <div>
                    @foreach (TagViewModel tag in item.Tags)
                    {
                        <span class="badge rounded-pill bg-bleu-ca text-white me-1 p-2">@tag.Libelle</span>
                    }
                </div>
            </div>
        </a>
    }
}
```



```

}
else if (!string.IsNullOrEmpty(Model.Recherche))
{
    <h3 class="text-center mt-4">Aucun résultat trouvé</h3>
}

@if (!string.IsNullOrEmpty(Model.Recherche))
{
    @* BTN CREATION *@
    <a class="btn btn-primary" asp-action="CreationDefinition" asp-route-libelle="@Model.Recherche">
        Ajouter une définition pour "@Model.Recherche"
    </a>
}

```

Cette page est l'entrée sur l'application *Lexigle*. Elle permet à l'utilisateur de rechercher une définition déjà existante et elle affiche une liste des dernières définitions ajoutées ou modifiées.

- **Structure et logique Razor** : La vue utilise des conditions Razor (@if, @else if) pour adapter l'affichage de l'interface. Si aucun mot n'est recherché, la page affiche uniquement la barre de recherche et la liste des ajouts récents. Si une recherche est lancée, le code boucle (@foreach) sur les résultats trouvés.
- **Tag Helpers et liaison** : Pour le bouton de création qui apparaît en cas de recherche, j'ai utilisé les Tag Helpers **asp-action** et **asp-route-libelle**. Cela permet de générer proprement l'URL de redirection vers le formulaire de création sans écrire de liens en dur (*hardcoding*), facilitant ainsi la maintenance des routes.
- **Composants Bootstrap** : J'ai utilisé les composants de cartes (card, card-body) et des badges (badge) pour structurer la présentation des définitions de manière claire, moderne et responsive.

Formulaire de création

```

@model CreationDefinitionViewModel

@{
}

<form asp-action="CreerDefinition" method="post">

    <div data-controller="switch-btn" class="position-relative">
        @*LIBELLE*@
        <div class="mb-2">
            <label asp-for="Libelle" class="form-label">Libellé</label>
            <input type="text" asp-for="Libelle" class="form-control" placeholder="Libellé" data-switch-btn-target="libelle"/>
        </div>

        @*ACRONYME*@
        <div class="mb-2">
            <label asp-for="Acronyme" class="form-label">Acronyme</label>
            <input asp-for="Acronyme" type="text" class="form-control" placeholder="Acronyme" data-switch-btn-target="acronyme" />
        </div>

        @* BUTTON SWITCH *@
        <button type="button" class="btn btn-secondary position-absolute" style="top: 71px; left: -60px;" data-action="click->switch-btn#switch">6#8644</button>
    </div>

    @*DESCRIPTION*@
    <div class="mb-2">
        <label asp-for="Description" class="form-label">Description</label>
        <textarea asp-for="Description" class="form-control" placeholder="Rédiger la description ici.."></textarea>
    </div>

    <div class="mt-3 d-flex justify-content-end">
        <input type="submit" name="valider" value="Soumettre ma proposition" class="btn btn-primary" />
    </div>

</form>

@* TAGS *@
<div class="mb-2">
    <label asp-for="Tags" class="form-label">Mots-clés</label>
    <select asp-for="Tags" data-controller="tags-tom-select" class="form-control" placeholder="Mots-clés" multiple></select>
</div>

```

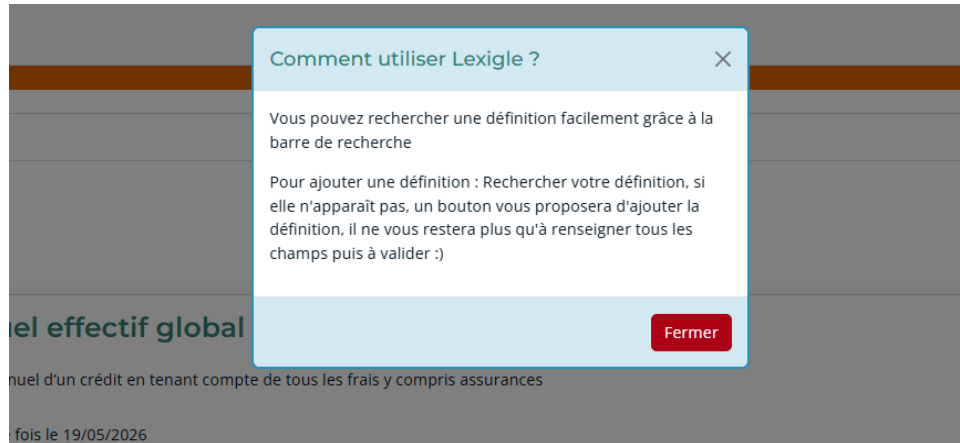
Plutôt que d'utiliser des attributs name classiques, chaque champ de saisie (<input>, <textarea>) utilise le Tag Helper **asp-for**. Il lie directement l'élément HTML à une propriété du modèle de données (ex: asp-for="Description"). Cela automatise la génération des attributs « id » et « name », assure la cohérence avec le

modèle et prépare l'affichage des messages d'erreur en cas de mauvaise saisie.

Le formulaire est balisé avec `<form>`, et utilise **asp-action="CreerDefinition"** pour définir la méthode du contrôleur qui réceptionnera les données lors du clic sur le bouton de soumission (`type="submit"`).

La modal d'information

Une des façons d'éviter la création de doublons en base de données, le bouton d'ajout d'une définition n'est pas accessible directement depuis la page d'accueil. L'utilisateur doit obligatoirement effectuer une recherche au préalable. Afin de garantir une bonne expérience utilisateur (UX) et d'expliquer cette démarche, j'ai intégré une modal d'information sur la page d'accueil.



Côté code, ce composant modal utilise le framework CSS **Bootstrap** pour la structure et l'animation graphique.

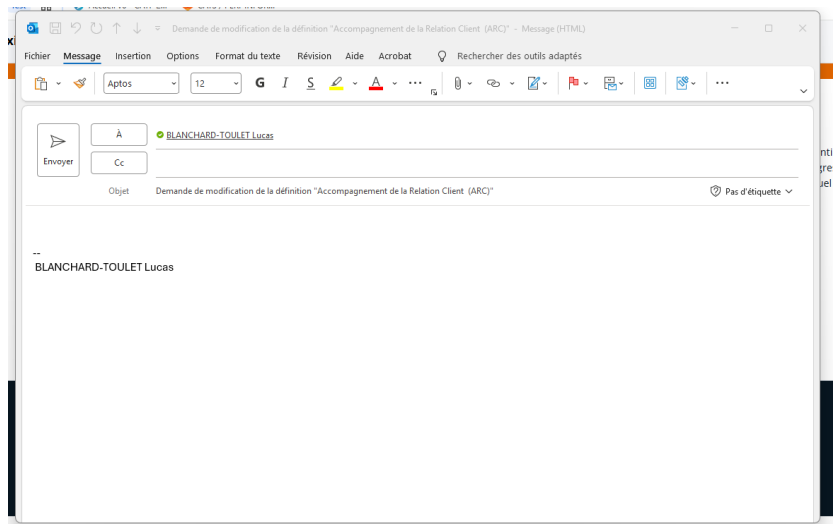
```
@* Information Modal *@
<div>
  <partial name="_ModalInfo" />
</div>
```

```
@{
}
<button type="button" class="btn btn-outline-info rounded-circle position-absolute" style="top: 120px; right: 225px" data-bs-toggle="modal" data-bs-target="#infoModal">
  <span class="fa-solid fa-info"></span>
</button>

<div class="modal fade" id="infoModal" tabindex="-1">
  <div class="modal-dialog modal-info">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Comment utiliser Lexigle ?</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        <p>Vous pouvez rechercher une définition facilement grâce à la barre de recherche</p>
        <p>Pour ajouter une définition : Rechercher votre définition, si elle n'apparaît pas, un bouton vous proposera d'ajouter la définition, il ne vous restera plus qu'à renseigner tous les champs puis à valider :)</p>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-danger" data-bs-dismiss="modal">Fermer</button>
      </div>
    </div>
  </div>
</div>
```

Pour maintenir un code propre, structuré et respectant le principe de réutilisabilité, j'ai encapsulé ce bloc HTML dans une **Vue Partielle (Partial View)** d'ASP.NET Core Razor. L'appel se fait simplement via le Tag Helper `<partial name="_ModalInfo" />`. Cela permet d'isoler le code de la modal dans un fichier indépendant, allégeant la vue principale et facilitant sa maintenance ou sa future réutilisation sur d'autres pages du site.

L'envoi d'e-mail pour suggérer une modification



```
@{
    string sujet = Uri.EscapeDataString($"Demande de modification de la définition \"{Model.Libelle} ({Model.Acronyme})\"");
}

<p>Par : @Model.Agent</p>
<p>Vous pouvez suggérer une modification en envoyant un mail à<br />
<a href="mailto:@Model.Email?subject=@sujet">@Model.Email<span class="fa-solid fa-envelope ms-1"></span></a>
</p>
```

Cette fonctionnalité permet de contacter l'agent responsable d'une définition via son client de messagerie natif (ex. : Outlook).

- **Intégration statique** : L'ouverture de la messagerie est gérée côté client par le navigateur à l'aide d'une simple balise d'ancrage `<a>` et du protocole standard `mailto:`. Cela évite le développement d'un service d'envoi SMTP lourd sur l'application.
- **Génération dynamique des paramètres (UX)** : Les attributs du lien sont injectés par le moteur de rendu Razor :
 - `@Model.Email` : Définit l'adresse du destinataire.
 - `subject=@sujet` : Préremplit l'objet du mail avec le libellé et l'acronyme de la définition.
- **Encodage de l'URL** : Pour rendre la chaîne conforme aux normes HTTP et éviter de casser le lien HTML avec les espaces ou caractères spéciaux du titre, le sujet est encodé côté serveur en C# à l'aide de la méthode `Uri.EscapeDataString()`.

Développer la partie dynamique des interfaces web (CP4)

La recherche : les suggestions autocomplétées

Ar	Rechercher
Vente par enregistrement	
ARC	
ARM	

Pour rendre la recherche intuitive, j'ai créé en JavaScript un menu déroulant dynamique suggérant des définitions en fonction de la recherche.

```
/* Recherche - Stimulus Controller
 */
import { Controller } from "/lib/stimulus/dist/stimulus.min.js"

export default class extends Controller {

  static targets = ["input", "suggestionBox"];
  static values = { url: String }

  suggestion() {
    console.log(this.urlValue)
    //Annule le minuteur précédent s'il existe
    clearTimeout(this.debounceTimer)

    //Si trop court on vide les suggestions et on s'arrête
    if (query.length < 2) {
      this.suggestionBoxTarget.innerHTML = ''
      return
    }

    //Lance un minuteur, la requête se déclenche seulement si l'utilisateur n'a pas retaper dans les 300ms
    this.debounceTimer = setTimeout(() => {

      //Encoder la saisie pour éviter des problèmes avec les caractères spéciaux
      fetch(`${this.urlValue}?query=${encodeURIComponent(query)}`)
      //Convertir la réponse HTTP en JSON
      .then(res => res.json())
      .then(data => {
        //Vide les anciennes suggestions avant d'afficher les nouvelles
        this.suggestionBoxTarget.innerHTML = ''

        data.forEach(s => {
          const li = document.createElement('li')
          li.classList.add('list-group-item', 'list-group-item-action')
          li.textContent = s

          //Au clic on remplit l'input avec la suggestion et ferme la liste
          li.addEventListener('click', () => {
            this.inputTarget.value = s
            this.suggestionBoxTarget.innerHTML = ''
            this.inputTarget.closest('form').submit()
          })
          this.suggestionBoxTarget.appendChild(li)
        })
      })
    }, 300)
  }
}
```

Le contrôleur Stimulus appelle un endpoint (une route du contrôleur qui appelle une méthode du contrat pour récupérer tous les Libellés, Acronymes, et tags), et le contrôleur Stimulus est appelé dans la vue par les attributs HTML data- .

```

/// <summary>
/// Utiliser par l'appel fetch dans stimulus, pour generer des suggestions en fonction de ce que tape l'utilisateur.
/// </summary>
/// <param name="query">La chaine de caractères saisie dans la barre de recherche</param>
/// <returns>
/// Une liste JSON correspondant à la saisie ou une liste vide si la saisie est inférieure à 2 caractères
/// </returns>
[HttpGet("Suggestions")]
public async Task<IActionResult> Suggestions(string query)
{
    if (string.IsNullOrEmpty(query) || query.Length < 2)
    {
        return Json(new List<string>());
    }

    ICollection<string?> suggestions = await _domaine.GenererSuggestionsRecherche(query);
    return Json(suggestions);
}

```

```

@* BARRE DE RECHERCHE *@
<form asp-action="Rechercher" method="get" class="mt-4">
    <div class="row g-2">
        <div class="col position-relative" data-controller="recherche" data-recherche-url-value="@Url.Content("~/Lexigle/Suggestions")">
            <input type="text" asp-for="Recherche" class="form-control form-control-lg" data-recherche-target="input" data-action="input->recherche#suggestion" />
            <ul data-recherche-target="suggestionBox" class="list-group position-absolute w-100 z-3 shadow"></ul>
        </div>
        <div class="col-auto">
            <button type="submit" class="btn btn-lg btn-primary">
                Rechercher
            </button>
        </div>
    </div>
</form>

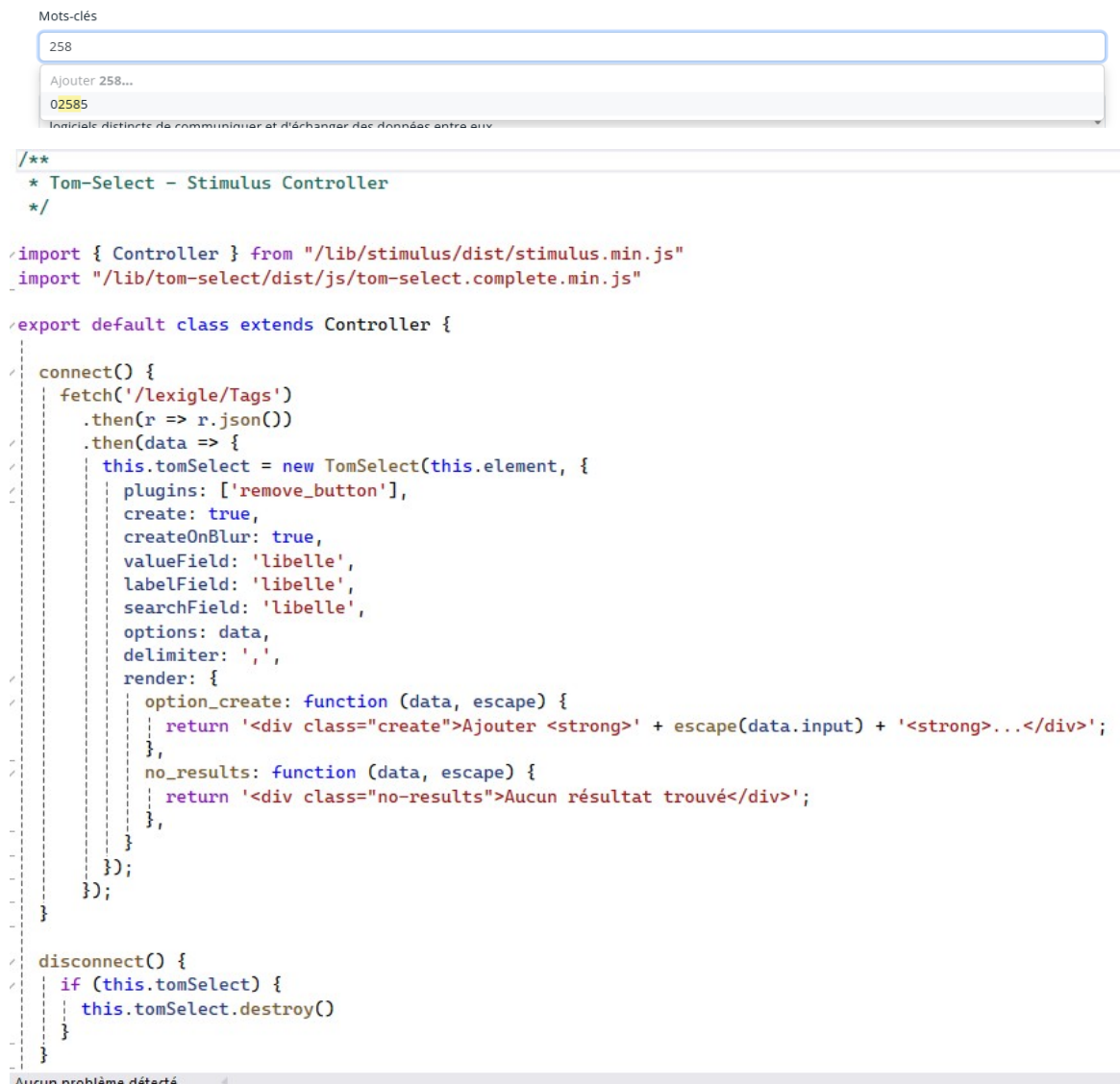
```

Pour bien comprendre la mécanique de cette recherche, voici le détail du traitement de la donnée entre le client (navigateur) et le serveur (.NET) :

1. **L'événement (Front)** : Chaque saisie dans le champ HTML déclenche le contrôleur Stimulus.
2. **Le Debounce (Optimisation)** : Si l'utilisateur tape le mot "ARM", Stimulus ne lance pas 3 requêtes HTTP. Le minuteur annule les requêtes précédentes tant que l'utilisateur tape vite (temporisation de 300ms). Une seule requête part une fois la saisie stabilisée.
3. **La promesse (Asynchronisme)** : La fonction `fetch()` émet une requête HTTP GET asynchrone vers l'URL `/Lexigle/Suggestions?query=...`. Pendant que le serveur traite la demande, le navigateur n'est pas bloqué ; et l'utilisateur peut continuer à interagir avec la page.
4. **Le traitement Serveur (Back)** : L'action du contrôleur C# reçoit la chaîne, appelle le service, sérialise le résultat en tableau **JSON** et renvoie un code HTTP 200 OK.
5. **La mise à jour du DOM (Front)** : La promesse du `fetch` est résolue (`.then()`). Le JavaScript boucle sur le tableau JSON, génère dynamiquement des balises `<li>` à la volée avec les données reçues et les injecte dans le nœud HTML parent (la `suggestionBox`).

L'ajout des mots-clés avec Tom Select

La gestion des mots-clés exige une interface intuitive. Pour implémenter cette saisie interactive, j'ai opté pour la bibliothèque Tom Select, encapsulée dans un contrôleur JavaScript Stimulus qui gère l'association des tags à leur définition.



Fonctionnement technique de l'algorithme Front-End :

1. **Hydratation asynchrone (fetch)** : Dès la connexion du contrôleur (`connect()`), le script interroge l'endpoint `/Lexigle/Tags` pour récupérer au format JSON l'ensemble des mots-clés unique de la base. (filtrés en amont côté serveur grâce à l'utilisation de `.Distinct()`).
2. **Saisie prédictive (Autocomplétion)** : Tom Select propose à l'utilisateur les tags existants au fil de sa saisie pour l'aiguiller et standardiser les mots-clés du dictionnaire.
3. **Création à la volée (create: true)** : Si le tag saisi n'existe pas encore, l'option permet à l'utilisateur de le générer instantanément.
4. **Ergonomie (plugins: ['remove_button'])** : Ce plugin injecte une croix de suppression pour retirer rapidement un tag sélectionné par erreur.
5. **Format d'envoi** : Lors de la soumission du formulaire, les mots-clés sélectionnés

sont sérialisés et renvoyés sous forme d'une chaîne de caractères unique, délimitée par des virgules (delimiter: ', ').

Flash messages dynamiquement et externalisés

AffichageResultatsRecherche_Error	Une erreur est survenue lors de la recherche
Definition_creation_e	Une erreur est survenue lors de la création de la définition
Definition_creation_succes	La définition a été créée avec succès
Definition_deja_existante	La définition que vous essayez de creer existe déjà
Definition_modification_error	Une erreur est survenue lors de la modification de la définition
Definition_modification_succes	La définition a été modifiée avec succès
Definition_obtenir_error	Une erreur inattendue est survenue lors du chargement de la définition.
Delete_error	Une erreur est survenue lors de la suppression de la définition
Delete_succes	Supprimer avec succès
Erreur403	Vous n'avez pas les autorisations nécessaires pour accéder à cette ressource.
Erreur404	La ressource demandée est introuvable.
Erreur5XX	Une erreur interne est survenue sur le serveur.
ErreurGenerique	Une erreur est survenue.
Error_chargement_page	Une erreur est survenue lors du chargement de la page
User_non_autorise	Vous n'avez pas l'autorisation d'accéder à cette ressource.

```
// Message de succès  
_flash.Success(FlashMsg.Definition_creation_succes);
```

Environnement de développement

La définition a été créée avec succès

×

Rechercher

Les récents ajouts

Application Programming Interface (API)

Pour enrichir l'expérience utilisateur (UX), l'application génère des notifications temporaires adaptées à ses actions (succès, échec). Cette fonctionnalité orchestre trois couches de l'application :

1. **La Source (Fichier .resx)** : Les messages sont centralisés dans un fichier de ressources sous forme de propriétés fortement typées (ex: `FlashMsg.Definition_creation_succes`). Cela évite les chaînes de caractères codées en dur (*magic strings*) et sécurise la maintenance.
2. **Le Contrôleur (C# / ASP.NET)** : Selon le résultat du traitement métier (try/catch), le contrôleur intercepte l'action et injecte dynamiquement la ressource choisie dans le service de notification via `_flash.Success()`. Le message est stocké temporairement en session pendant la redirection.
3. **L'Interface (Rendu Razor & Bootstrap)** : Au chargement, la vue vérifie la session. Si un message existe, Razor génère dynamiquement le bloc HTML en y injectant les classes Bootstrap adaptées (`alert-success` ou `alert-danger`) pour afficher le bandeau d'alerte contextuel.

| Mettre en place une base de données relationnelle (CP5)

Conception initiale et évolution

La conception de **Lexigle** a évolué pour s'adapter à la réalité du terrain et aux délais d'un stage de deux mois.

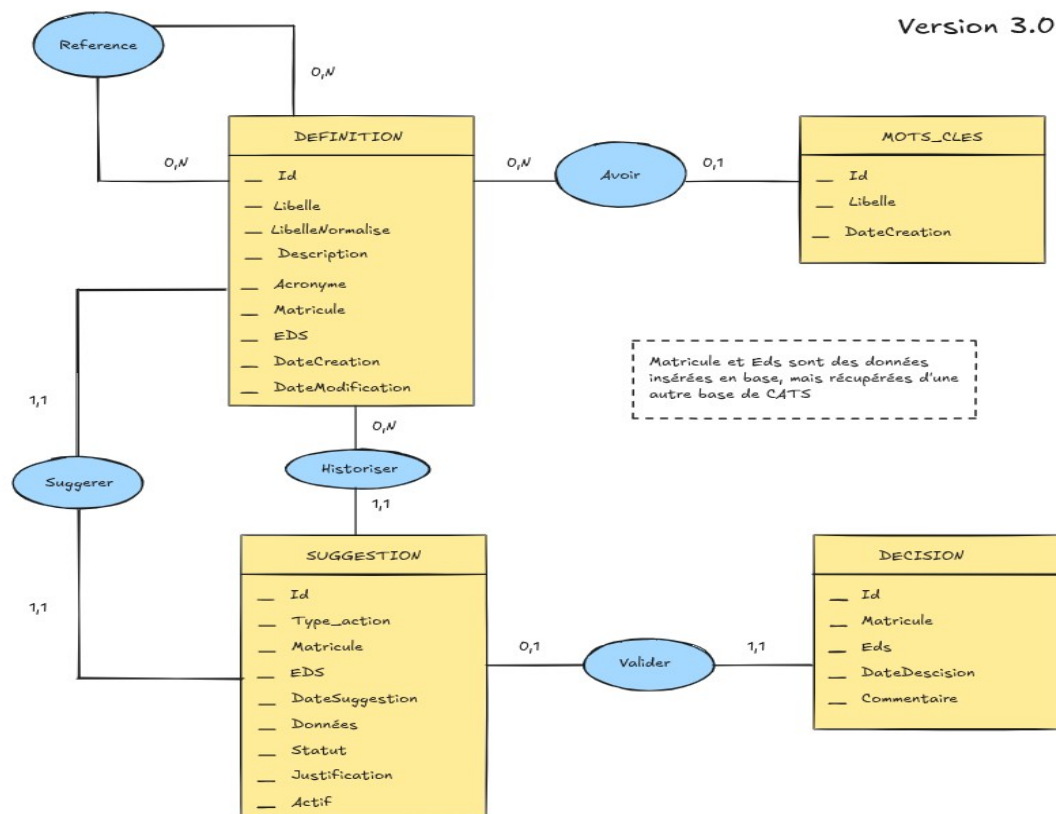
Initialement, j'avais modélisé une table distincte pour les **Acronymes**, liée à la table **Définitions** par deux relations :

- Une relation **One-to-Many** pour l'acronyme principal.
- Une relation **Many-to-Many** pour gérer des acronymes secondaires.

Après deux semaines de développement, nous avons constaté que cette structure complexifiait inutilement le code pour un besoin métier très rare. Pour débloquer le projet et avancer sur le **Lot 1**, nous avons pris la décision pragmatique de supprimer la table Acronyme et d'intégrer l'information directement dans la table **Définition**.

Conception retenue

Cette simplification m'a permis de stabiliser la base de données et de me concentrer sur des fonctionnalités à plus forte valeur ajoutée, comme les liaisons entre les termes.



Le schéma final s'organise autour de **4 entités** :

- **L'entité Définition** : Contient le libellé, l'acronyme et les détails du créateur de la définition.
 - Une relation **Many-to-Many** de Définition vers Définition : permet de lier des définitions entre elles, que ce soit pour pointer vers un synonyme ou pour regrouper des termes qui partagent un même contexte métier.
 - Une relation **One-To-Many** de Définition vers Tag : permet d'ajouter des mots clés pour la recherche et le filtrage.
- **L'entité Tag (Mots-clés)** : Contient un mot clé et une référence externe pour pointer sur la définition.
 - Une relation **Many-To-One** de Tag vers Définition. J'ai délibérément opté pour une structure **ManyToOne** (Chaque tag est physiquement lié à une seule et unique définition) pour trois raisons pragmatiques :
 1. **Sécurité des données (Zéro effet de bord)** : Si un utilisateur modifie ou corrige l'orthographe d'un tag, le changement reste isolé sur sa définition et n'impacte pas le reste du dictionnaire.
 2. **Maintenance simplifiée (Cascade Delete)** : La suppression d'une définition nettoie automatiquement ses tags en base de données, évitant de devoir gérer la suppression complexe de tags orphelins.
 3. **Respect des deadlines** : Ce modèle a considérablement simplifié les requêtes LINQ et accéléré le développement en fin de stage, avec un impact totalement négligeable sur le stockage.
- **L'entité Suggestion** : Pour proposer la modification d'une définition, si elle est validée, elle est appliquée à la définition. Les suggestions ne sont pas supprimées et servent ainsi d'historique des versions des définitions.
 - Une relation **Many-to-One** de Suggestion vers Définition
 - Une relation **One-to-One** de Suggestion vers Décision
- **L'entité Décision** : Elle permet de stocker et d'afficher le retour de l'administrateur (validation ou motif de refus) pour chaque suggestion soumise.

Persistances et migrations

La couche de **Persistence** est constituée de classes C# (souvent appelées entités) qui représentent de manière fidèle la structure des futures tables de la base de données. Ces classes servent ensuite de point d'appui à l'ORM pour générer les scripts de création via le mécanisme des migrations.

L'exemple ci-dessous illustre la classe de persistance associée à l'entité *Definition* :

```
[Table("Definition")]
public class Definition
{
    [Key]
    public int Id { get; set; }

    [MaxLength(128)]
    public string Libelle { get; set; } = default!;

    public string LibelleNormalise { get; set; } = default!;

    public string Description { get; set; } = default!;

    [StringLength(10, MinimumLength = 2)]
    [RegularExpression("^[A-Z0-9]+$")]
    public string? Acronyme { get; set; } = default!;

    public string Matricule { get; set; } = default!;

    public string Eds { get; set; } = default!;

    public DateTime DateCreation { get; set; } = default!;

    public DateTime DateModification { get; set; } = default!;

    //Navigation

    public ICollection<Definition> DefinitionsReferenceParent { get; set; } = [];

    public ICollection<Definition> DefinitionsReferenceEnfant { get; set; } = [];

    public ICollection<Tag> Tags { get; set; } = [];
}
```

Pour lier ces objets C# à la base de données, j'ai utilisé l'ORM **Entity Framework Core (EF Core)**. La configuration des tables se fait directement au sein du code grâce aux **data annotations** (ou attributs) visibles au-dessus des propriétés :

- **[Table("Definition")]** : Indique explicitement à l'ORM que la table physique en base de données doit être nommée « Definition ».
- **[Key]** : Spécifie que la propriété qui suit (ici Id) fait office de clé primaire auto-incrémentée.
- **[MaxLength(X)]**, **[StringLength(x,y)]** et **[RegularExpression(x)]** : Permettent de contraindre le schéma de la base de données, par exemple en définissant une taille maximale pour les chaînes de caractères.
- **L'opérateur de nullabilité (?)** : Introduit en C#, il indique à EF Core si la colonne correspondante en base de données accepte les valeurs nulles (NULL) ou si elle

doit être configurée comme NOT NULL.

Enfin, les propriétés regroupées sous le bloc de « Navigation » permettent de mapper les relations et les clés étrangères (un-à-plusieurs ou plusieurs-à-plusieurs) entre cette table et les autres entités du système, garantissant ainsi l'intégrité référentielle des données.

Pour orchestrer l'ensemble de ces entités et faire le lien avec le serveur de base de données, il m'a fallu configurer le `DbContext`. Ce fichier, qui hérite du `DbContext` d'Entity Framework Core, joue le rôle de passerelle centrale de l'application pour la persistance des données.

Son implémentation s'articule autour de deux responsabilités majeures :

```
#region Lexigle
```

```
public DbSet<Definition> Definitions { get; set; } = default!;
```

```
public DbSet<Tag> Tag { get; set; } = default!;
```

```
#endregion Lexigle
```

```
/**
 * Definitions
 */
modelBuilder.Entity<Definition>(definition =>
{
    definition.HasIndex(def => def.Libelle);
    definition.HasIndex(def => def.Acronyme);

    definition.HasMany(def => def.DefinitionsReferenceEnfant)
        .WithMany(def => def.DefinitionsReferenceParent)
        .UsingEntity(j => j.ToTable("DefinitionReference"));

    definition.HasMany(def => def.Tags)
        .WithOne()
        .HasForeignKey(t => t.DefinitionId);
});

/**
 * Tags
 */
modelBuilder.Entity<Tag>(tags =>
{
    tags.HasIndex(t => t.Libelle);
});
```

Les migrations sont générées par ligne de commande, `dotnet ef migrations add nomDeLaMigration` ce qui génère un script de migration, et un fichier de snapshot. Puis la migration est appliquée grâce à la commande `dotnet ef database update`.

Développer des composants d'accès aux données SQL (CP6)

Extraction et requêtage asynchrone (Lecture)

Pour récupérer les informations en base de données, j'ai développé des méthodes d'accès exploitant l'ORM **Entity Framework Core** à travers des requêtes LINQ.

```
public async Task<ICollection<DefinitionServiceModel>> RechercherDefinition(string recherche)
{
    string rechercheNormalisee = NormaliserText.Normalise(recherche);

    return await _context.Definitions
        .Include(def => def.Tags)
        .Where(def => def.LibelleNormalise.Contains(rechercheNormalisee)
            || (def.Acronyme != null && def.Acronyme.Contains(rechercheNormalisee))
            || def.Tags.Any(t => t.Libelle.Contains(rechercheNormalisee)))
        .OrderByDescending(def => def.Acronyme == rechercheNormalisee
            || def.LibelleNormalise == rechercheNormalisee
            || def.Tags.Any(t => t.Libelle == rechercheNormalisee))
        .ThenByDescending(def => def.LibelleNormalise.StartsWith(rechercheNormalisee)
            || (def.Acronyme != null && def.Acronyme.StartsWith(rechercheNormalisee))
            || def.Tags.Any(t => t.Libelle.StartsWith(rechercheNormalisee)))
        .ThenBy(def => def.DateModification)
        .Select(def => def.ToServiceModel())
        .ToListAsync() ?? [];
}
```

- **Optimisation et asynchronisme** : La méthode utilise le mot-clé `async` / `await` associé à `ToToListAsync()`. En déléguant le temps d'attente à la base de données de manière asynchrone, le serveur reste disponible pour traiter d'autres requêtes, améliorant ainsi les performances globales de l'application.
- **Chargement lié** : Grâce à l'instruction `.Include(def => def.Tags)`, je force la récupération immédiate des tags associés à chaque définition en une seule requête SQL (jointure), évitant ainsi le problème de performance classique des requêtes N+1.
- **Projection vers la couche métier** : fin de la requête, l'instruction `.Select(def => def.ToServiceModel())` convertit directement les entités de persistance brutes issues de la base de données en objets `DefinitionServiceModel`. Cela garantit que la couche d'infrastructure ne renvoie jamais d'entités SQL directes aux couches supérieures.

Du modèle métier vers l'affichage (Présentation)

Une fois les objets métier (ServiceModel) récupérés par le système, ils doivent être adaptés aux contraintes de l'interface graphique avant d'être envoyés à la vue.

```
public static DefinitionViewModel ToViewModel(this DefinitionServiceModel definition)
{
    return new DefinitionViewModel
    {
        Id = definition.Id,
        Libelle = definition.Libelle,
        Description = definition.Description,
        Acronyme = definition.Acronyme,
        Matricule = definition.Matricule,
        Eds = definition.Eds,
        Tags = [..definition.Tags.Select(t => t.ToViewModel())],
        DateCreation = definition.DateCreation,
        DateModification = definition.DateModification
    };
}
```

La méthode d'extension ToViewModel() intervient à la suite de la recherche. Elle prend le DefinitionServiceModel pour instancier un DefinitionViewModel unique, contenant strictement les données requises par l'affichage Razor.

Traitement des flux d'écriture (Création d'une définition)

Après avoir traité la lecture, l'application doit gérer l'envoi de nouvelles données. Pour le scénario de création d'une définition, le flux de mapping et de contrôle d'accès des données s'articule ainsi :

A. Le mapping initial (Interface → Service) :

```
public static CreationDefinitionServiceModel ToServiceModel(this CreationDefinitionViewModel definition)
{
    return new CreationDefinitionServiceModel
    {
        Libelle = definition.Libelle,
        Description = definition.Description,
        Acronyme = definition.Acronyme,
        Tags = [..definition.Tags.Select(t => new CreationTagsServiceModel { Libelle = t})],
    };
}
```

Dès la soumission du formulaire, la méthode d'extension ToServiceModel() intercepte le ViewModel pour le projeter vers un modèle de service.

B. Le contrôle d'unicité en base de données :

```
public async Task<bool> VerifierDefinitionExistante(string libelle, string? acronyme, int? id = null)
{
    return await _context.Definitions
        .Where(def => def.Libelle == libelle && def.Acronyme == acronyme && (!id.HasValue || def.Id != id))
        .AnyAsync();
}
```

Avant de valider l'insertion, ce composant d'accès effectue une requête optimisée via AnyAsync(). Ce choix technique permet de demander au SGBD de vérifier l'existence d'un doublon (sur le libellé ou l'acronyme) de manière ultra-rapide, renvoyant un simple booléen sans charger l'entité complète en mémoire.

C. La projection finale vers la persistance :

```
public static Definition ToPersistance(this CreationDefinitionServiceModel definition)
{
    DateTime now = DateTime.Now;
    return new Definition()
    {
        Libelle = definition.Libelle,
        LibelleNormalise = NormaliserText.Normalise(definition.Libelle),
        Description = definition.Description,
        Acronyme = definition.Acronyme?.ToUpperInvariant(),
        Matricule = definition.Matricule,
        Eds = definition.Eds,
        DateCreation = now,
        DateModification = now,
    };
}
```

Une fois les contrôles validés, la méthode `ToPersistance()` convertit le modèle de service en entité SQL `Definition`. C'est à cette étape précise que j'injecte les données système d'audit (`DateTime.Now`) et que je force la mise en majuscules de l'acronyme (`ToUpperInvariant()`) pour ensuite les enregistrer en base.

```
public async Task<int> CreerDefinition(CreationDefinitionServiceModel creation_sm)
{
    Definition definition = creation_sm.ToPersistance();

    definition.Tags = [... creation_sm.Tags.Select(t => t.ToPersistance())];

    _context.Definitions.Add(definition);

    await _context.SaveChangesAsync();

    return definition.Id;
}
```

Traitement des flux de mise à jour (Modification d'une définition)

Le scénario de modification réutilise des principes similaires, mais introduit un composant d'accès aux données spécifique pour la mise à jour des relations.

A. Requête d'extraction pour modification :

```
public async Task ModifierDefinition(ModificationDefinitionServiceModel modif_sm)
{
    //Récupérer la définition à modifier
    Definition definition = await _context.Definitions
        .Include(def => def.Tags)
        .FirstOrDefaultAsync(def => def.Id == modif_sm.Id) ?? throw new KeyNotFoundException("La définition avec l'ID {id} n'existe pas. ");

    //Appliquer la modification
    modif_sm.Modification(definition);

    //Sauvegarder en base de données
    await _context.SaveChangesAsync();
}
```

Pour modifier une entité, le composant d'accès doit d'abord extraire l'enregistrement existant du SGBD. J'utilise `FirstOrDefaultAsync()` combiné à un `Include(def => def.Tags)` pour charger la définition et ses mots-clés associés dans le contexte de tracking d'Entity Framework. Si l'ID n'est pas trouvé, une exception `KeyNotFoundException` est immédiatement levée.

B. Le mapping de modification et purge des relations :

```
public static void Modification(this ModificationDefinitionServiceModel modif_sm, Definition definition)
{
    definition.Id = modif_sm.Id;
    definition.Libelle = modif_sm.Libelle;
    definition.LibelleNormalise = NormaliserText.Normalise(modif_sm.Libelle);
    definition.Description = modif_sm.Description;
    definition.Acronyme = modif_sm.Acronyme?.ToUpperInvariant();
    definition.Matricule = modif_sm.Matricule;
    definition.Eds = modif_sm.Eds;
    definition.DateModification = DateTime.Now;

    //Suppression et recréation des tags
    definition.Tags.Clear();
    definition.Tags = [.. modif_sm.Tags.Select(t => t.ToPersistance())];
}
```

C'est le point critique de la mise à jour. Modifier des listes liées (les tags) avec un ORM peut générer des conflits d'index. Pour sécuriser l'accès aux données, la méthode `Modification` applique un comportement strict : la collection de tags existante est vidée via `Clear()` avant de réassigner la nouvelle liste de tags convertis.

Développer des composants métier côté serveur (CP7)

1. Gestion dynamique des paramètres métiers et sécurisation (Le Vault)

Dans le cadre d'une application bancaire d'entreprise, la flexibilité et la sécurité des configurations sont critiques. Plutôt que de figer des variables métier dans le code source ou de dépendre de fichiers de configuration locaux statiques, j'ai mis en place une architecture d'externalisation dynamique s'appuyant sur un **Vault** hautement sécurisé.

Les constantes métier (comme le nombre maximal de dernières définitions à afficher ou le seuil de troncature des descriptions) sont stockées de manière centralisée dans un coffre-fort numérique (Vault).

```
    "NB_LAST_DEF": "10",  
    "NB_CHAR_DESC": "200"  
}
```

Pour manipuler ces configurations au sein du code C# de manière fortement typée et centralisée, j'ai rajouté dans la classe existante de constantes ConfigurationsKeys qui fait office de dictionnaire technique pour l'application :

```
namespace A02E7Lexigle.WebApp.Properties;  
  
public static class ConfigurationsKeys  
{  
    #region # PRIVATIF  
    [REDACTED]  
    [REDACTED]  
    public const string NbDerniereDef = "NB_LAST_DEF";  
    public const string MaxLenthTroncate = "NB_CHAR_DESC";  
}
```

Au lancement de l'application, l'initialisation de ces options métier est déléguée à une couche de configuration dédiée (LexigleStartup), appelée directement dans le point d'entrée du serveur :

```
#region labtp  
// Options métier de l'application  
LexigleStartup.SettingUp(builder);  
// Add Domaine  
DomaineStartup.SettingUp(builder);  
// Add FlashMessageService  
MessagesFlashStartup.SettingUp(builder);  
// Add API AgentEds  
AgentEdsStartup.SettingUp(builder);  
// Add API Habilitation  
HabilitationStartup.SettingUp(builder);  
// Add Tracer  
TracerStartup.SettingUp(builder);  
// Add Mailing  
MailingStartup.SettingUp(builder);  
#endregion labtp
```


La méthode `Setup` récupère les configurations stockées dans le Vault. Pour sécuriser le lancement de l'application et éviter qu'elle ne fonctionne avec des données incorrectes ou manquantes, j'ai mis en place une validation stricte. Si une clé indispensable est absente du coffre-fort, le serveur lève immédiatement une exception `InvalidOperationException` et bloque le démarrage de l'application.

```
using A02E7Lexigle.WebApp.Properties;

namespace A02E7Lexigle.WebApp.Startup;

public static class LexigleStartup
{
    public static void Setup(WebApplicationBuilder builder)
    {
        //Récupération des paramètres pour truncate et pour le nombre de dernières définitions souhaité.
        string? nb_derniereDef = builder.Configuration[ConfigurationsKeys.NbDerniereDef];
        string? valeur_truncate = builder.Configuration[ConfigurationsKeys.MaxLenthTruncate];

        //Validation du nombre de derniere def
        if (string.IsNullOrEmpty(nb_derniereDef))
        {
            throw new InvalidOperationException("Aucune valeur de nombre de definition choisi n'est définie dans la configuration");
        }

        //Validation de la valeur de maxLenth pour Truncate
        if (string.IsNullOrEmpty(valeur_truncate))
        {
            throw new InvalidOperationException("Aucune valeur de maxLenth choisi pour tronquer les définitions n'est définie dans la configuration");
        }

        //Configuration des options de Lexigle
        builder.Services.Configure<LexigleOptions>(options =>
        {
            options.NbDerniereDef = int.Parse(nb_derniereDef);
            options.MaxLenthTruncate = int.Parse(valeur_truncate);
        });
    }
}
```

Une fois validées, ces chaînes de caractères sont converties en types numériques (`int.Parse`) et injectées dans un objet de configuration global.

```
namespace A02E7Lexigle.WebApp.Properties;

public class LexigleOptions
{
    public int NbDerniereDef { get; set; } = default!;

    public int MaxLenthTruncate { get; set; } = default!;
}
```

Grâce à cette architecture, si les équipes métier du Crédit Agricole décident de passer l'affichage de 10 à 20 définitions ou d'ajuster la troncature du texte, la modification est appliquée en temps réel dans le Vault. **Aucune recompilation du code C#, seulement le déploiement de package (pipeline CI/CD) est relancé**, garantissant une disponibilité maximale de l'application.

2. Injection de dépendances et isolation des responsabilités

Pour garantir un couplage faible et une testabilité maximale de mon code, le `LexigleController` utilise exclusivement l'injection de dépendances par constructeur. Il ne crée jamais lui-même ses outils, mais reçoit des abstractions (interfaces) qui lui sont automatiquement fournies et préparées en arrière-plan par le framework .NET dès le démarrage de l'application.

```

namespace A02E7Lexigle.WebApp.Controllers;

[Route("lexigle")]
[Authorize(Roles = UserRoles.UTILISATEUR)]
public class LexigleController(
    IDomaineService domaineService,
    ITracerService tracerService,
    IFlashMessageService msgFlashService,
    IAgentEdsClient agentEds,
    IOptions<LexigleOptions> lexigleOptions) : Controller
{
    private readonly IDomaineService _domaine = domaineService;
    private readonly ITracerService _tracer = tracerService;
    private readonly IFlashMessageService _flash = msgFlashService;
    private readonly IAgentEdsClient _agentEds = agentEds;

    private readonly int _nbDerniereDef = lexigleOptions.Value.NbDerniereDef;
    private readonly int _maxLengthTroncate = lexigleOptions.Value.MaxLengthTroncate;
}

```

Dans ce constructeur, on identifie clairement la séparation stricte des responsabilités de notre architecture :

- IDomaineService : Encapsule toute la logique métier et l'accès sécurisé aux données (requêtes LINQ du CP6).
- ITracerService : Gère la journalisation applicative et l'audit technique (Logs).
- IFlashMessagesService : Permet de pousser des notifications visuelles asynchrones à l'utilisateur.
- IOptions<LexigleOptions> : Reçoit les configurations dynamiques préalablement extraites du Vault. Les valeurs locales _nbDerniereDef et _maxLengthTroncate sont ainsi hydratées de façon sécurisée dès l'instanciation du contrôleur.

3. Traitement et sécurisation d'une action serveur (Scénario de modification)

Pour illustrer l'orchestration des composants métiers par la couche Web, analysons la méthode ModifierDefinition exécutée lors de la réception d'un formulaire en HTTP POST :

```

/// <summary>
/// Reçoit les données modifier du view model, vérifie si l'auteur est le même qu'à la création,
/// </summary>
/// <param name="modif_vm">les données stocké dans le view model.</param>
/// <returns>Au formulaire de modification si un Exception est levée, sinon à l'index après modi
[HttpPost("definition/modifier")]
public async Task<ActionResult> ModifierDefinition(ModificationDefinitionViewModel modif_vm)
{
    //Essaie de la vérification des correspondance d'auteurs
    try
    {
        //Permet de récupérer l'auteur de la définition d'origine
        DefinitionServiceModel definition_sm = await _domaine.ObtenirDefinition(modif_vm.Id);

        //Puis vérifier que les auteurs correspondent
        if (!VerifierAuteur(definition_sm))
        {
            _flash.Danger(FlashMsg.User_non_autorise);
            return RedirectToAction(nameof(Rechercher));
        }
    }
    catch(Exception ex)
}

```

Sécurité d'accès métier (Vérification multi-niveaux de l'auteur) : La première action critique du serveur consiste à récupérer l'enregistrement d'origine en base de données afin de valider les droits réels de l'utilisateur connecté. Pour cela, le contrôleur fait appel à la méthode privée `VerifierAuteur`. Cette fonction implémente trois barrières de sécurité successives :

1. **Propriété directe** : Elle vérifie si le matricule de l'agent connecté correspond à celle du créateur initial de la définition.
2. **Super-utilisateur** : Elle accorde le droit de modification si l'utilisateur possède le rôle d'Administrateur Métier (ADMIN_METIER).
3. **Rattaché à la structure** : Elle autorise la modification si l'agent appartient à la même entité/structure EDS que l'auteur d'origine.

Si aucun de ces critères n'est validé, la méthode renvoie `false`. Le contrôleur interrompt immédiatement le flux, génère une alerte de sécurité via le service de messages flash et redirige l'utilisateur pour bloquer toute tentative de modification illégitime.

```
// Si echec validation
if (!ModelState.IsValid)
{
    // Messages flash
    _flash.Warning(FlashMsg.Definition_creation_e);
    ModelState.Values.SelectMany(v => v.Errors).ToList().ForEach(e => _flash.Warning(e.ErrorMessage));

    // Stockage du ViewModel pour le récupérer après la redirection
    this.SetTempData(modif_vm);

    // Redirection vers le formulaire de modification
    return RedirectToAction(nameof(ModificationDefinition), new { id = modif_vm.Id });
}
```

Validation des contraintes de surface (ModelState) : Le serveur vérifie ensuite la conformité des données saisies (champs requis, longueur des textes). En cas d'échec de validation, le contrôleur intercepte la liste des erreurs, les pousse dynamiquement dans le service de messages flash pour avertir l'utilisateur, et recharge le formulaire en lui renvoyant ses saisies initiales via l'objet temporaire `TempData`.

```
//Verification que les champs renseigner ne correspondent pas déjà à une défintion
try
{
    if (await _domaine.VerifierDefinitionExistante(modif_vm.Libelle, modif_vm.Acronyme, modif_vm.Id))
    {
        throw new Exception();
    };
}
catch (Exception)
```

Contrôle d'unicité et prévention des doublons : Avant de persister les modifications, le contrôleur sollicite le domaine métier pour s'assurer que les nouvelles valeurs (libellé ou acronyme) ne provoquent pas de doublon avec une autre définition existante. Si la méthode `VerifierDefinitionExiste` renvoie un booléen positif, une exception est levée, l'incident est tracé dans les logs applicatifs et l'action est annulée.

```

//Tentative de modification de la définition
try
{
    ModificationDefinitionServiceModel modif_definition_sm = modif_vm.ToServiceModel();

    string matricule = this.User.GetMatricule();
    string eds = ObtenirEdsParMatricule(matricule).Result;

    modif_definition_sm.Matricule = matricule;
    modif_definition_sm.Eds = eds;

    // modification de la définition
    await _domaine.ModifierDefinition(modif_definition_sm);

    // Message de succès
    _flash.Success(FlashMsg.Definition_modification_succes);
}
catch (Exception ex)
{
    // Log
    Log.Error(ex.Message);

    // Trace
    await _tracer.Trace(TypeTrace.Erreur, string.Format(LogMsg.Definition_modification_error, modif_vm.Id, ex.Message));

    // Message flash
    _flash.Danger(FlashMsg.Definition_modification_error);
}

// Redirection vers l'index
return RedirectToAction(nameof(Rechercher));

```

Persistance finale et gestion des pannes (try / catch) : Une fois toutes les barrières de contrôle franchies, le contrôleur enrichit le modèle de service avec les données d'audit de l'agent (matricule et structure EDS) avant de valider définitivement la mise à jour en base de données. L'ensemble de ce flux est encapsulé dans une structure try / catch : si un incident d'infrastructure survient (ex: coupure réseau avec le SGBD), l'erreur technique est journalisée précisément via _tracer pour les équipes de maintenance du Crédit Agricole, tandis qu'un message sécurisé prévient l'utilisateur du dysfonctionnement.

4. Traitement textuel standardisé

Afin de garantir la fiabilité des recherches et d'empêcher les doublons en base de données causés par les variations de saisie, j'ai développé un composant utilitaire centralisé qui applique une chaîne de nettoyage stricte sur les chaînes de caractères.

```

namespace A02E7Lexigle.Domaine.Helper;
public static partial class NormaliserText
{
    public static string SupprimerAccents(string texte)
    {
        string normalized = texte.Normalize(NormalizationForm.FormD);
        return new string(normalized
            .Where(c => CharUnicodeInfo.GetUnicodeCategory(c) != UnicodeCategory.NonSpacingMark)
            .ToArray())
            .Normalize(NormalizationForm.FormC);
    }

    public static string Normalise(string texte)
    {
        string sansAccents = SupprimerAccents(texte);
        string sansSpeciaux = MyRegexCaractereSpeciaux().Replace(sansAccents, " ");
        return MyRegexEspaceMultiple().Replace(sansSpeciaux, " ").Trim().ToUpperInvariant();
    }

    [GeneratedRegex(@"\s+")]
    private static partial Regex MyRegexEspaceMultiple();

    [GeneratedRegex(@"^[a-zA-Z0-9\s]")]
    private static partial Regex MyRegexCaractereSpeciaux();
}

```


Pour standardiser les saisies et fiabiliser le moteur de recherche, le composant applique un traitement articulé autour de deux méthodes complémentaires. La première, `SansAccents`, se charge d'isoler et de supprimer les caractères accentués en exploitant la normalisation Unicode pour ne reconstruire qu'un texte pur.

La méthode principale, `Normalise`, orchestre ensuite le nettoyage final. Elle s'appuie sur deux expressions régulières optimisées via l'attribut **[GeneratedRegex]** : l'une remplace les caractères spéciaux par des espaces, tandis que la seconde purge les espaces doubles. Le traitement se conclut par un `.Trim()` éliminant les espaces résiduels aux extrémités et un `.ToUpperInvariant()` pour figer le texte en majuscules.

Contrairement à une instanciation classique, l'attribut `[GeneratedRegex]` demande au compilateur de générer le code de la Regex dès la compilation. Cela supprime le temps d'analyse à l'exécution et économise les ressources mémoire du serveur lors des saisies utilisateur.

5. Validation de la qualité du code (Tests Unitaires xUnit)

Pour sécuriser le serveur contre les régressions logicielles et valider scientifiquement le comportement de nos composants, j'ai mis en place une suite de tests automatisés avec le framework **xUnit** et la bibliothèque de **Moq**.

La robustesse de l'algorithme de normalisation est validée de manière isolée en suivant le cycle standard **AAA (Arrange, Act, Assert)** :

```
public class NormaliseTexteTest
{
    [Theory]
    [InlineData("Jean-Pierre", "JEAN PIERRE")]
    [InlineData("jean-pierre", "JEAN PIERRE")]
    [InlineData("Jean Pierre", "JEAN PIERRE")]
    [InlineData("jean pierre", "JEAN PIERRE")]
    [InlineData("Jean pierre", "JEAN PIERRE")]
    [InlineData("Jean---Pierre", "JEAN PIERRE")]
    [InlineData(" JeAn- --PIeRre ", "JEAN PIERRE")]
    [InlineData("sql      server", "SQL SERVER")]
    [InlineData(" sql ", "SQL")]
    [InlineData("DÉPLOIEMENT", "DEPLOIEMENT")]
    [InlineData("déploiement", "DEPLOIEMENT")]
    [InlineData("fête", "FETE")]
    [InlineData("hawai", "HAWAI")]
    [InlineData("à", "A")]
    public void Normalise_RetourneTexteNormalisee(string input, string output)
    {
        string result = NormaliserText.Normalise(input);
        Assert.Equal(output, result);
    }
}
```

Validation : Le test injecte une chaîne complexe contenant des espaces multiples, des accents et des caractères spéciaux (" âcrônÿmê ! "). L'assertion `Assert.Equal` confirme que le composant retourne exactement le résultat attendu et uniformisé ("ACRONYME"), prouvant la fiabilité de l'algorithme.

```

[Fact]
public async Task CreerDefinition_TagEtLibelleNormalise()
{
    //Arrange
    CreationDefinitionServiceModel sm = new()
    {
        Libelle = "libellé de-test",
        Tags =
        [
            new CreationTagsServiceModel { Libelle = "Jean-Pierre" },
            new CreationTagsServiceModel { Libelle = "déploiement" }
        ],
        Description = $"Description_1",
        Eds = "02585",
        Matricule = $"A0{1 + 1:D5}",
    };

    //ACT
    await _service.CreerDefinition(sm);

    //ASSERT
    Tag tag1 = _context.Tag.First(t => t.Libelle == "JEAN PIERRE");
    Tag tag2 = _context.Tag.First(t => t.Libelle == "DEPLOIEMENT");

    Definition definition = _context.Definitions.First(d => d.Libelle == "libellé de-test");
    Assert.Equal("LIBELLE DE TEST", definition.LibelleNormalise);
}

```

```

[Fact]
public async Task ModifierDefinition_TagEtLibelleNormalise()
{
    //Arrange
    await GenererDefinition(1);

    ModificationDefinitionServiceModel sm = new()
    {
        Id = 1,
        Libelle = "test modifié",
        Tags = [
            new CreationTagsServiceModel { Libelle = "Jean-Pierre" },
            new CreationTagsServiceModel { Libelle = "déploiement" },
        ]
    };

    //Act
    await _service.ModifierDefinition(sm);

    //ASSERT
    Tag tag1 = _context.Tag.First(t => t.Libelle == "JEAN PIERRE");
    Tag tag2 = _context.Tag.First(t => t.Libelle == "DEPLOIEMENT");

    Definition definition = _context.Definitions.First(d => d.Id == 1);
    Assert.Equal("TEST MODIFIE", definition.LibelleNormalise);
}

```

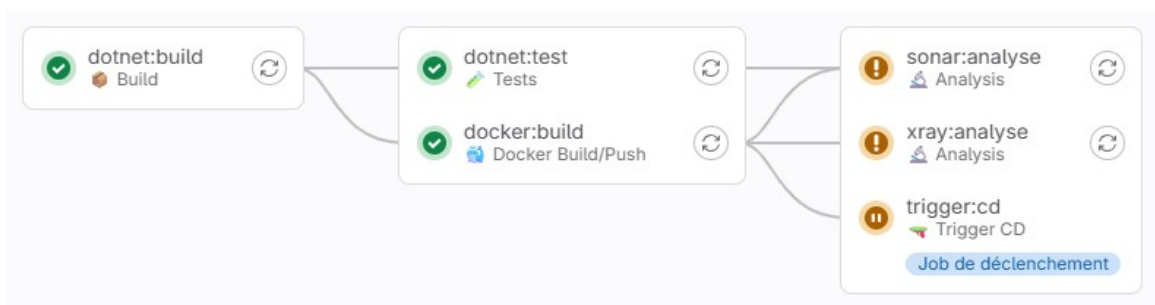
Ces deux tests automatisés (CreerDefinition_Normalise et ModifierDefinition_Normalise) vérifient que lors d'un ajout ou d'une mise à jour de définition, la couche Domaine intercepte correctement les données brutes fournies par l'utilisateur, applique la méthode Normalise sur le libellé ainsi que sur l'acronyme, et transmet uniquement l'entité nettoyée à la base de données. Les assertions confirment le succès de cette orchestration et garantissent l'absence de régression sur le traitement serveur.

Documenter le déploiement d'une application (CP8)

Bien que l'infrastructure de déploiement soit gérée en amont par les équipes du Crédit Agricole, j'ai été intégré au processus de livraison du Lot 1 de l'application. Accompagné par mon maître de stage, j'ai pu observer et analyser le fonctionnement de la chaîne d'automatisation industrielle (CI/CD) utilisée pour déployer l'application sur les environnements de Développement et de Recette (UAT).

1. L'Intégration Continue (CI)

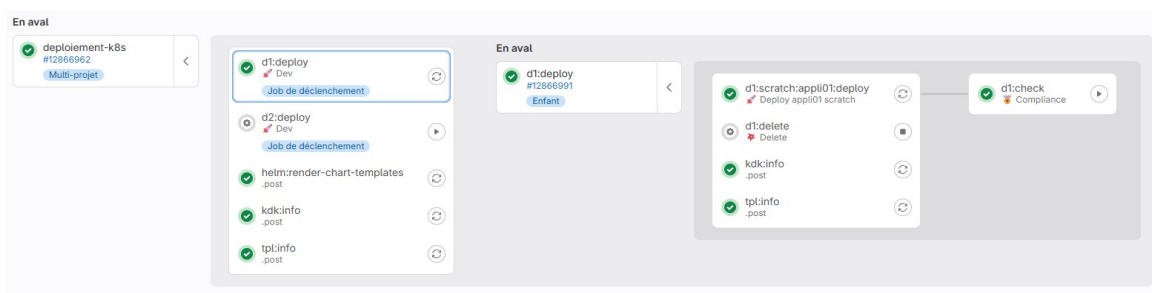
Le pipeline de CI s'exécute automatiquement à chaque push de code effectué sur le dépôt Git. J'ai ainsi pu suivre les étapes automatisées qui garantissent la qualité du serveur avant toute mise en production :



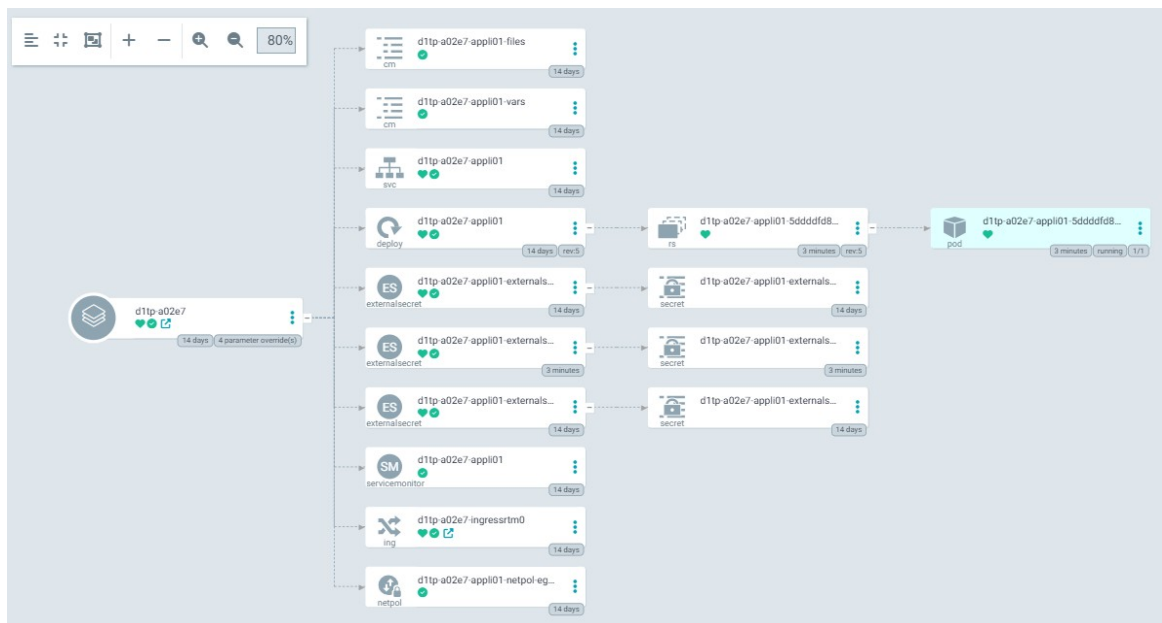
Le flux compile d'abord le code source (`dotnet:build`), puis lance la suite de tests unitaires xUnit que j'ai développée (`dotnet:test`). En parallèle, une image conteneurisée est générée (`docker:build`). Enfin, le code subit une analyse statique de qualité via **SonarQube** (`sonar:analyse`) ainsi qu'un scan de sécurité des dépendances (`xray:analyse`) avant de préparer le déclenchement du déploiement (`trigger:cd`).

2. Le Déploiement Continu (CD)

Pour des raisons de sécurité propres au secteur bancaire, la phase de CD n'est pas entièrement automatisée. Son déclenchement vers l'environnement ciblé fait l'objet d'une validation humaine et se lance manuellement en un clic.



Dès que la livraison est lancée, l'outil **ArgoCD** prend le relais pour installer automatiquement l'application sur les serveurs **Kubernetes** de la banque.



Cet écran fonctionne comme une autre sorte de pipeline visuel pour le déploiement. Chaque bloc représente un élément de l'application sur le serveur (les fichiers de configuration, les connexions réseau, la sécurité). Le fonctionnement est très visuels; si tous les voyants sont au vert cela signifie que l'application est bien installée et qu'elle fonctionne normalement. Si un bloc change de couleur, cela permet d'identifier immédiatement où se situe le problème sur le serveur.

CONTEXTE COMPLÉMENTAIRE : INITIATION À LA MIGRATION DE SITES (SYMFONY)

En parallèle du projet principal *Lexigle* développé en C#, j'ai été initié à un chantier secondaire concernant les SELI (Sites En Ligne Interactifs). Il s'agit de la gamme de sites vitrines du Crédit Agricole destinés au grand public, qui reposent sur le framework PHP **Symfony**.

1. Problématique d'infrastructure et contraintes de sécurité

Dans le cadre d'une modernisation de ses infrastructures, la direction informatique a entrepris de centraliser l'ensemble de ses applications (outils internes en C# et sites publics Symfony) sur ses serveurs les plus récents. La contrainte technique majeure résidait dans la configuration de ces nouveaux serveurs sécurisés : ils imposaient l'utilisation de **Symfony 7.4** au minimum.

C'est au démarrage de ce projet que j'ai été confronté aux réalités de la sécurité au sein d'un grand groupe bancaire. À la suite d'un renouvellement du parc informatique de l'équipe, les machines récentes et les règles strictes du proxy réseau central bloquaient l'installation du framework ainsi que le téléchargement des packages PHP requis.

2. Travaux réalisés et suspension du chantier

J'ai pu tout de même amorcer la phase préparatoire de la migration :

- **Analyse** : Étude des alertes de dépréciation (*deprecations*) générées par le code actuel.
- **Nettoyage des dépendances** : Identification et suppression des packages obsolètes ou inutilisés.
- **Préparation technique** : Modification du fichier de configuration `composer.json` pour planifier la montée de version vers Symfony 7.4.

Ce chantier a finalement dû être suspendu en cours de route. Bien que je n'aie pas pu mener cette migration jusqu'à son terme, cette expérience m'a permis d'appréhender concrètement l'impact des politiques de sécurité réseau (proxy) sur le quotidien des équipes de développement, ainsi que la complexité liée à la gestion de l'obsolescence logicielle dans une grande entreprise.

Conclusion

Ce stage de deux mois au sein du Crédit Agricole a représenté une étape courte mais particulièrement intense et déterminante dans mon parcours de futur développeur. Cette expérience m'a plongé dans la réalité concrète de la production logicielle au sein d'une grande institution financière.

Un bilan projet formateur

Le bilan du développement de l'application *Lexigle* est très formateur. Sous la direction de mon maître de stage, l'accent a été mis dès le départ sur la rigueur technique plutôt que sur la quantité de fonctionnalités. J'ai donc dû intégrer les exigences de l'équipe : privilégier le "clean code", la sécurité et la mise en place systématique de tests à chaque étape, quitte à avancer plus lentement.

Grâce à cette méthodologie de travail, le lot 1 de l'application est entièrement finalisé et fonctionnel. Concernant le lot 2, la gestion des étiquettes (*tags*) est opérationnelle. Les fonctionnalités plus avancées — telles que la liaison des définitions entre elles, le système de suggestions et le panneau d'administration — n'ont pas pu être abordées par manque de temps.

Cependant, ce choix de la rigueur rend la base de l'application saine : la logique métier est sécurisée (habilitations, anti-doublons), l'algorithme de normalisation est optimisé pour le serveur et les paramètres de configuration clés ont été correctement externalisés. Cela rend l'outil facilement modifiable et maintenable à l'avenir par les administrateurs, sans avoir à retoucher au code source. Enfin, la pérennité du code est assurée par la suite de tests unitaires et d'intégration. Bien que j'aurais aimé aller plus loin dans les différents lots, je suis très satisfait de la fiabilité et de la propreté des modules que j'ai pu livrer.

Ce que ce stage m'a apporté

Sur le plan humain et organisationnel, ce stage m'a énormément appris sur le travail en équipe. Bien que j'ai été seul développeur sur *Lexigle*, j'ai constamment échangé avec mon maître de stage et d'autres collègues pour concevoir les fonctionnalités, appréhender les deadlines de production et comprendre les rouages, les atouts et les contraintes d'une structure d'envergure comme le Crédit Agricole.

La rigueur exigeante que m'a imposée mon maître de stage a été l'un des points les plus positifs de cette expérience : elle m'a forcé à perdre mes réflexes de développeur débutant pour adopter une posture professionnelle et méthodique.

Sur le plan technique, j'ai découvert le langage C# que je ne connaissais pas du tout. J'ai particulièrement apprécié la rigueur de sa syntaxe et les possibilités offertes par l'architecture multicouche. Cette immersion m'a également permis d'observer des outils industriels majeurs, de la gestion des secrets d'entreprise jusqu'aux chaînes de déploiement Kubernetes et ArgoCD. Enfin, l'initiation à la migration Symfony, bien que stoppée en plein vol à cause de contraintes matérielles liées au proxy réseau, m'a montré l'importance de savoir s'adapter aux imprévus du terrain.

Perspectives et projets d'avenir

Cette expérience a confirmé et renforcé ma passion pour la programmation orientée objet (POO) et le développement Back-End. À court terme, je vais développer un *side-project* personnel pour approfondir mes compétences en POO. J'ai choisi de le concevoir en Java, couplé à un framework Front-End comme React ou Angular.

À plus long terme et sur plusieurs années, j'ai l'ambition de devenir un développeur complet et polyvalent. J'envisage d'adapter mon projet personnel sur Android via Kotlin, de survoler le C++ à travers la création d'un jeu simple, et de réaliser un projet en C pour comprendre les rouages profonds des langages de bas niveau.

Pour concrétiser ces ambitions, mon objectif pour la rentrée de septembre est d'intégrer la Licence Professionnelle de développement à Poitiers en alternance. Ce cursus me permettra d'aller plus loin dans mon apprentissage théorique tout en continuant à renforcer mes compétences sur le terrain.